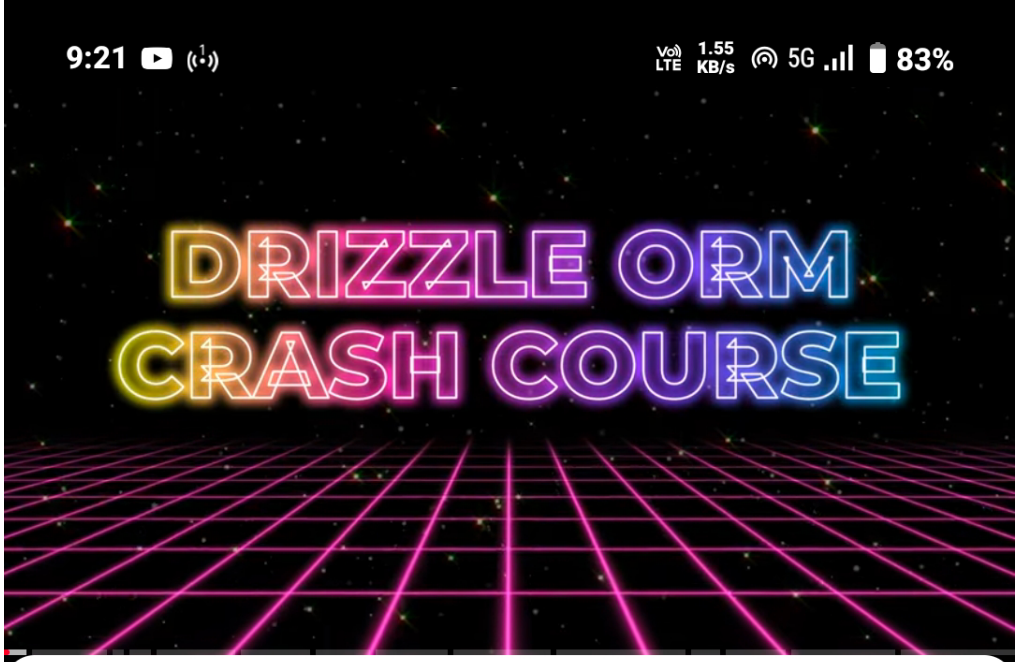


Drizzle ORM Full Course Tutorial For Begin...

@ThapaTechnical 6.7K views 7mo ago ...more

Join 199 Share

Comments 16

A Sir please create a full course of Dizzle and Prisma

Introduction to Drizzle ORM

- Drizzle ORM is a lightweight and type-safe Object Relational Mapping (ORM) library for SQL databases.
- It focuses on providing a simple and developer-friendly API while maintaining high performance.
- Drizzle ORM supports both TypeScript and JavaScript, offering full type safety for queries.
- Drizzle ORM provides built-in query building and executes queries wi relying on raw SQL strings.
- Designed for developers who want an ORM that's minimal, efficie to use.
- Even though Prisma is quite popular, we are going to use Drizzle course.

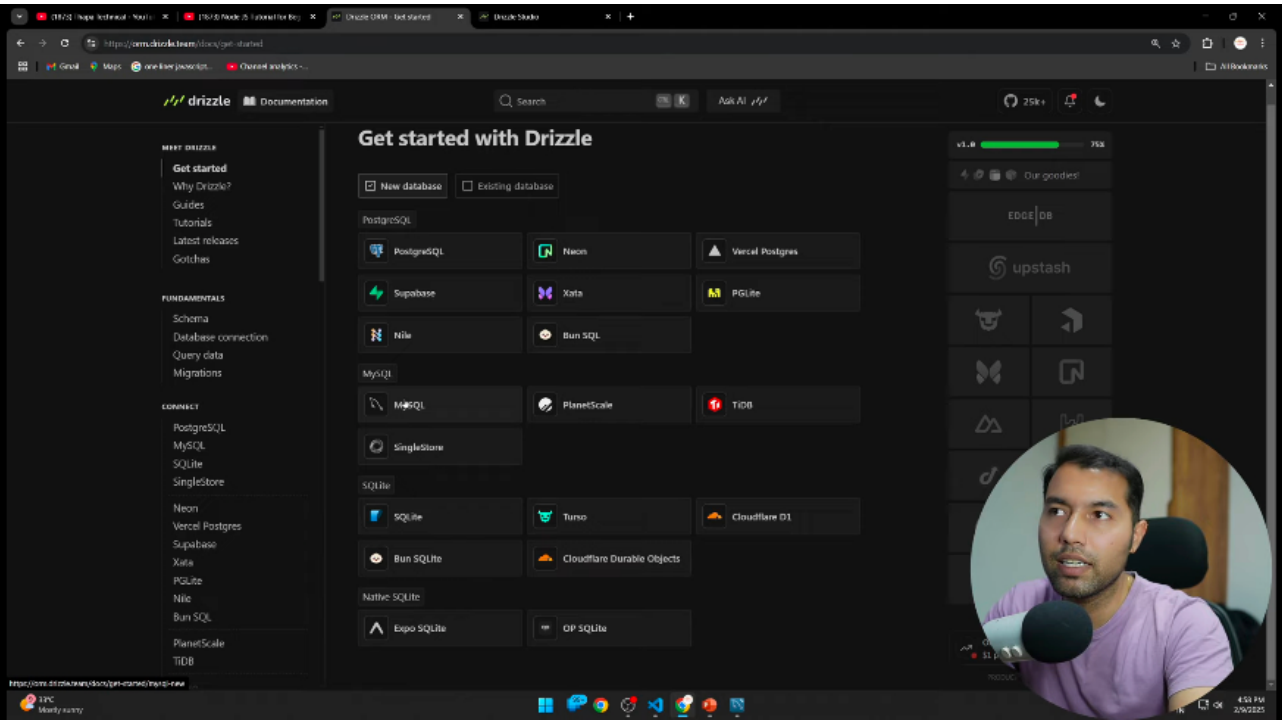
Why Drizzle ORM over Prisma?

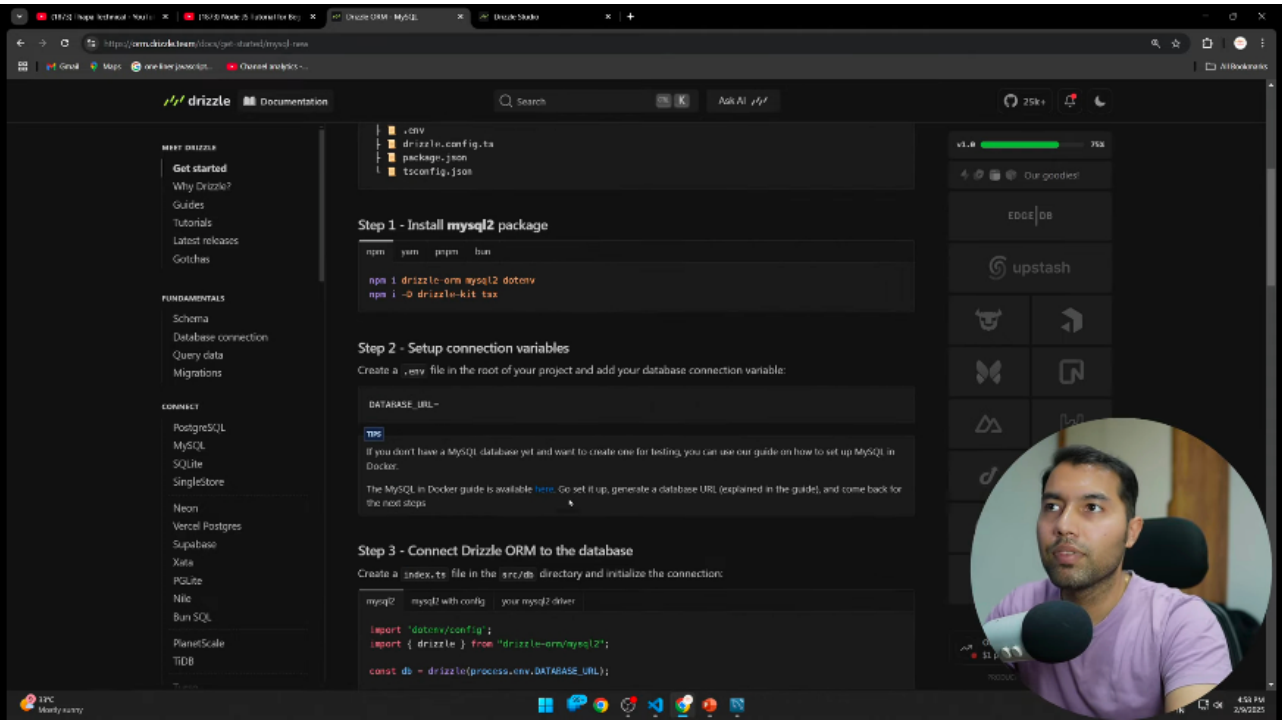
- Drizzle ORM is lightweight and doesn't rely on a Rust engine like Prisma, making it faster and more straightforward to use.
- Unlike Prisma, Drizzle ORM always executes a single query per request unless you're using its optional query API, which we recommend you avoid.
- Drizzle ORM is closer to raw SQL, so if you're familiar with SQL, you can easily translate SQL queries into Drizzle queries.
- All configurations in Drizzle ORM are done in JavaScript or TypeScript, unlike Prisma, which uses its own domain-specific language (DSL).
- Drizzle ORM is designed for simplicity and minimal overhead, allo write cleaner and more direct SQL-based code.
- Drizzle ORM's approach to queries and configurations makes it i developers who prefer a simple, SQL-native experience over con abstraction layers.

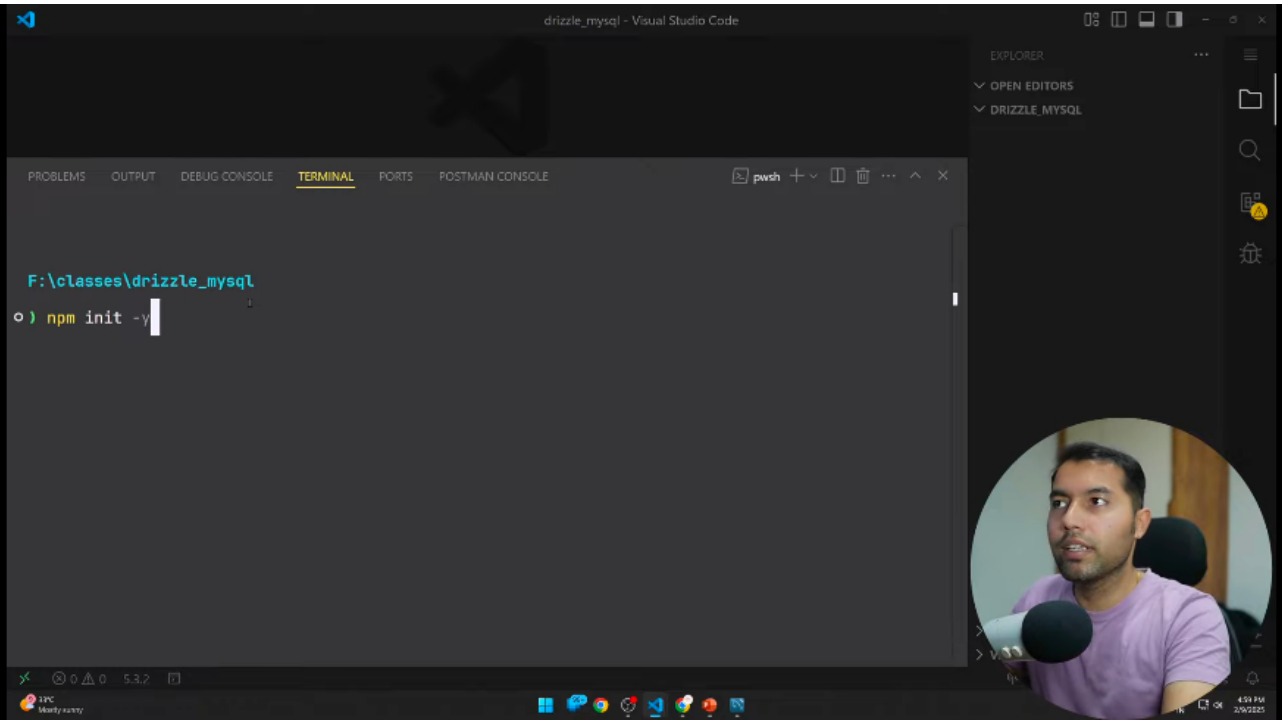
Comparison of SQL, Drizzle, and Prisma

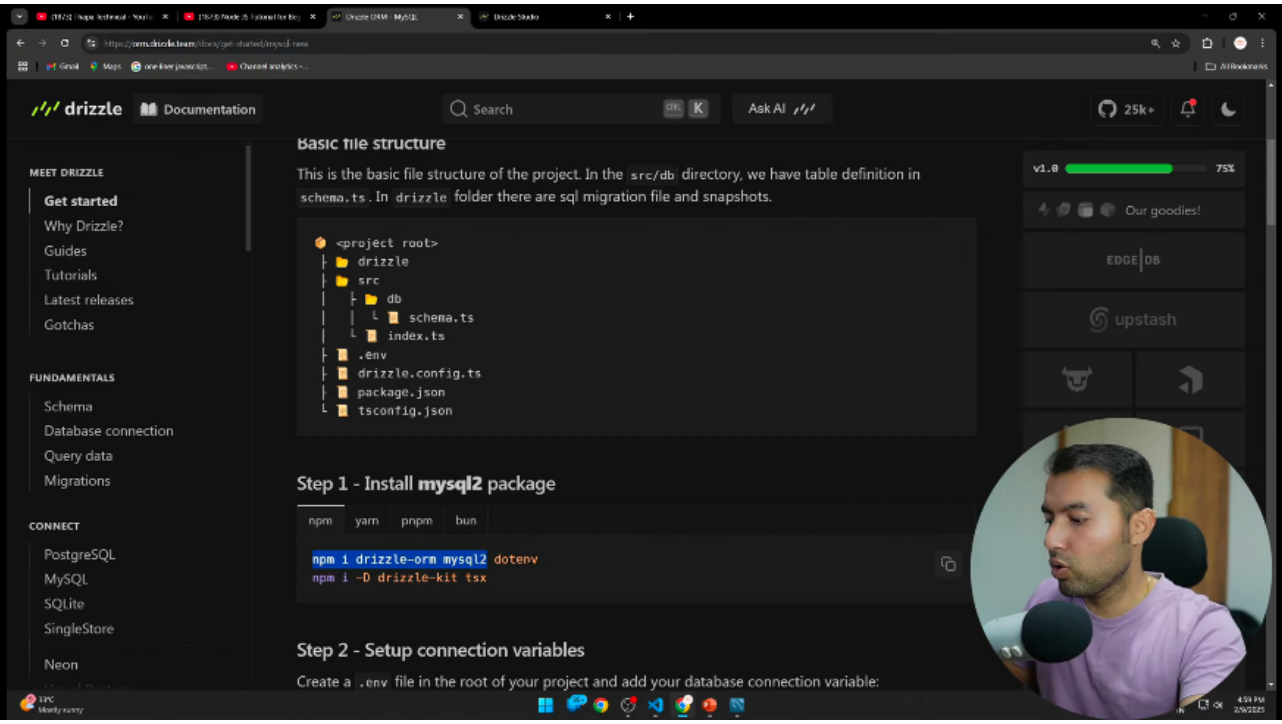
Comparison Between SQL, Prisma, and Drizzle

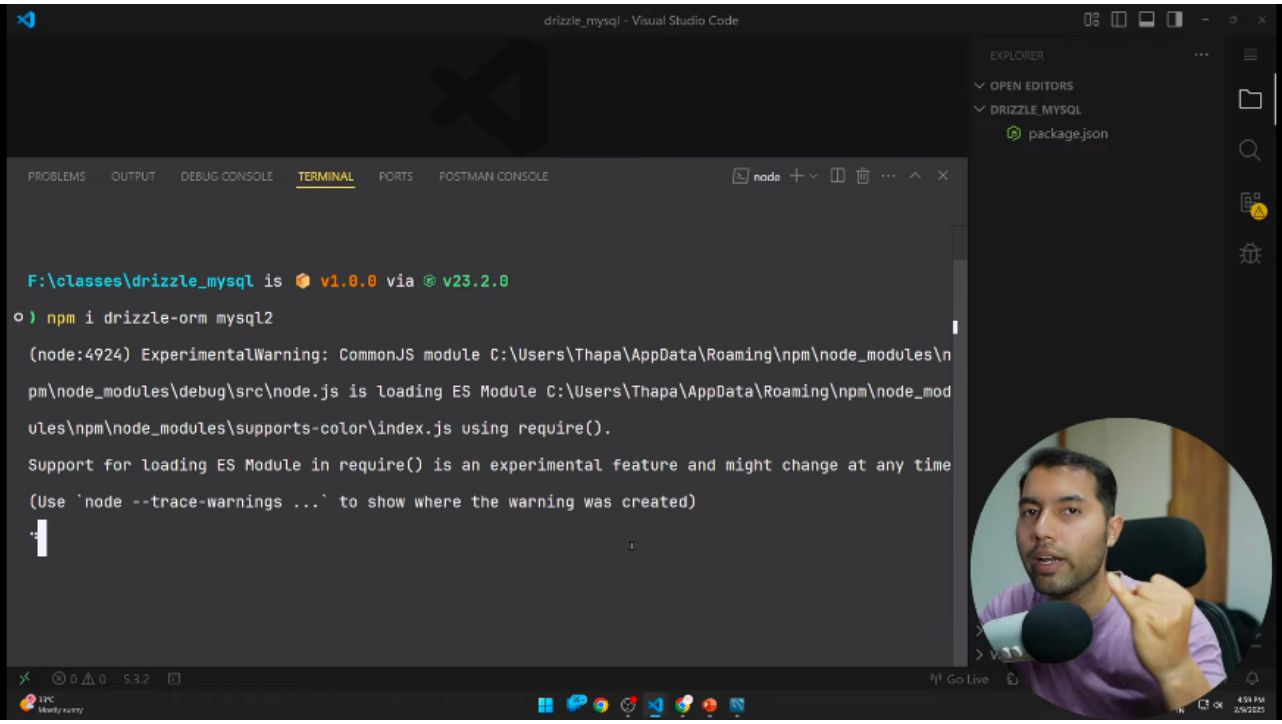
```
// SQL
SELECT * FROM users;

// Prisma
const users = await prisma.user.findMany();

// Drizzle
const users = await db.select().from('users');

// Drizzle with Query API - Please don't use
const users = await db.query.users.findMany()
```

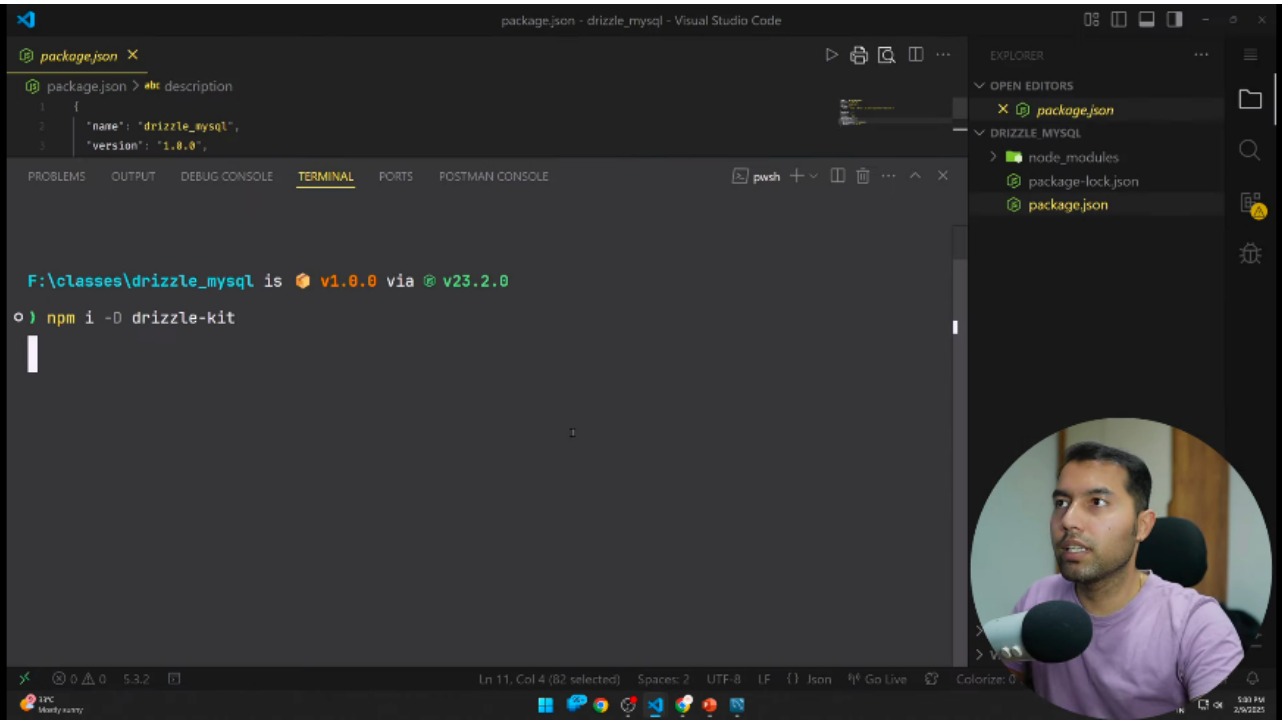

drizzle
Documentation
Search
Ask AI
Get started with Drizzle
New database
Existing database
PostgreSQL
PostgreSQL
Neon
Vercel Postgres
Supabase
Xata
PGLite
Nile
Bun SQL
MySQL
MySQL
PlanetScale
TiDB
SingleStore
SQLite
SQLite
Turso
Cloudflare D1
Bun SQLite
Cloudflare Durable Objects
Native SQLite
Expo SQLite
OP SQLite
Get started
Why Drizzle?
Guides
Tutorials
Latest releases
Gotchas
Schema
Database connection
Query data
Migrations
PostgreSQL
MySQL
SQLite
SingleStore
Neon
Vercel Postgres
Supabase
Xata
PGLite
Nile
Bun SQL
PlanetScale
TiDB
upstash

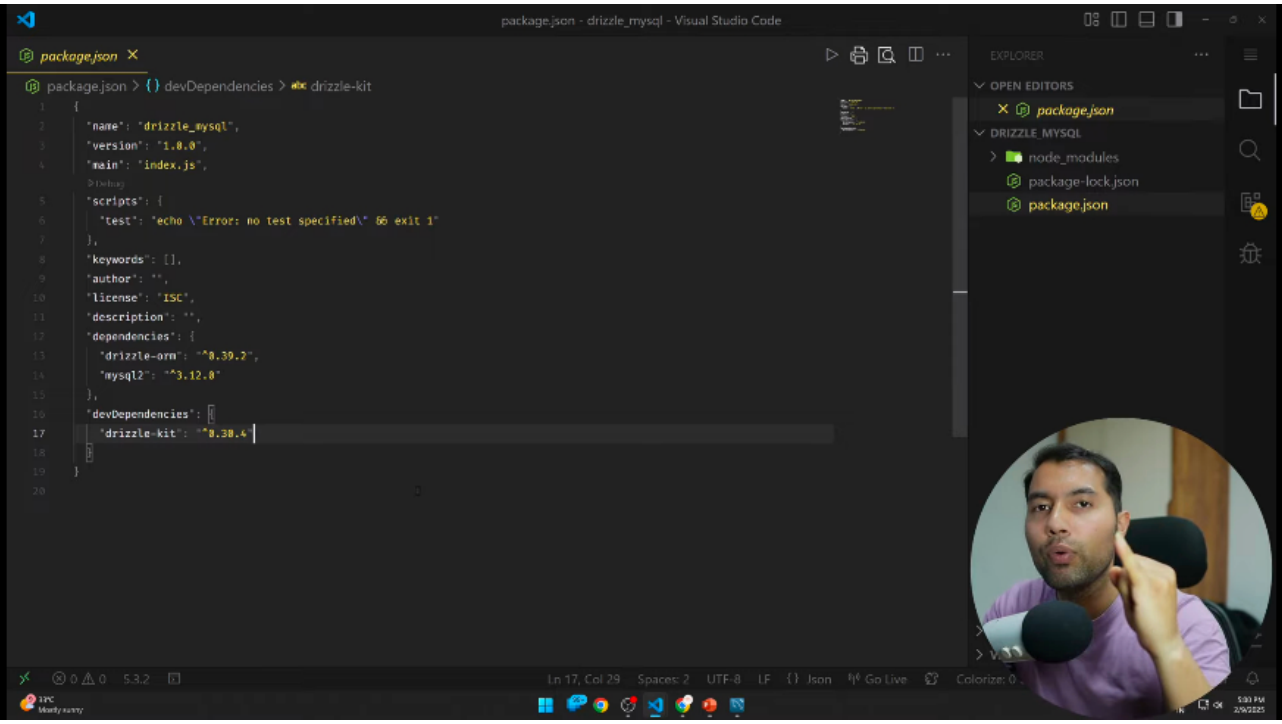

drizzle
Documentation
Search
Ask AI
25k+
MEET DRIZZLE
Get started
Why Drizzle?
Guides
Tutorials
Latest releases
Gotchas
FUNDAMENTALS
Schema
Database connection
Query data
Migrations
CONNECT
PostgreSQL
MySQL
SQLite
SingleStore
Neon
Vercel Postgres
Supabase
Xata
PGLite
Nile
Bun SQL
PlanetScale
TiDB
.env
drizzle.config.ts
package.json
tsconfig.json
Step 1 - Install mysql2 package
npm
yarn
pnpm
bun
npm i drizzle-orm mysql2 dotenv
npm i -D drizzle-kit tsx
Step 2 - Setup connection variables
Create a .env file in the root of your project and add your database connection variable:
DATABASE_URL=
TIPS
If you don't have a MySQL database yet and want to create one for testing, you can use our guide on how to set up MySQL in Docker.
The MySQL in Docker guide is available here. Go set it up, generate a database URL (explained in the guide), and come back for the next steps
Step 3 - Connect Drizzle ORM to the database
Create a index.ts file in the src/db directory and initialize the connection:
mysql2
mysql2 with config
your mysql2 driver
import 'dotenv/config';
import { drizzle } from "drizzle-orm/mysql2";
const db = drizzle(process.env.DATABASE_URL);
Our goodies!
EDGE DB
upstash

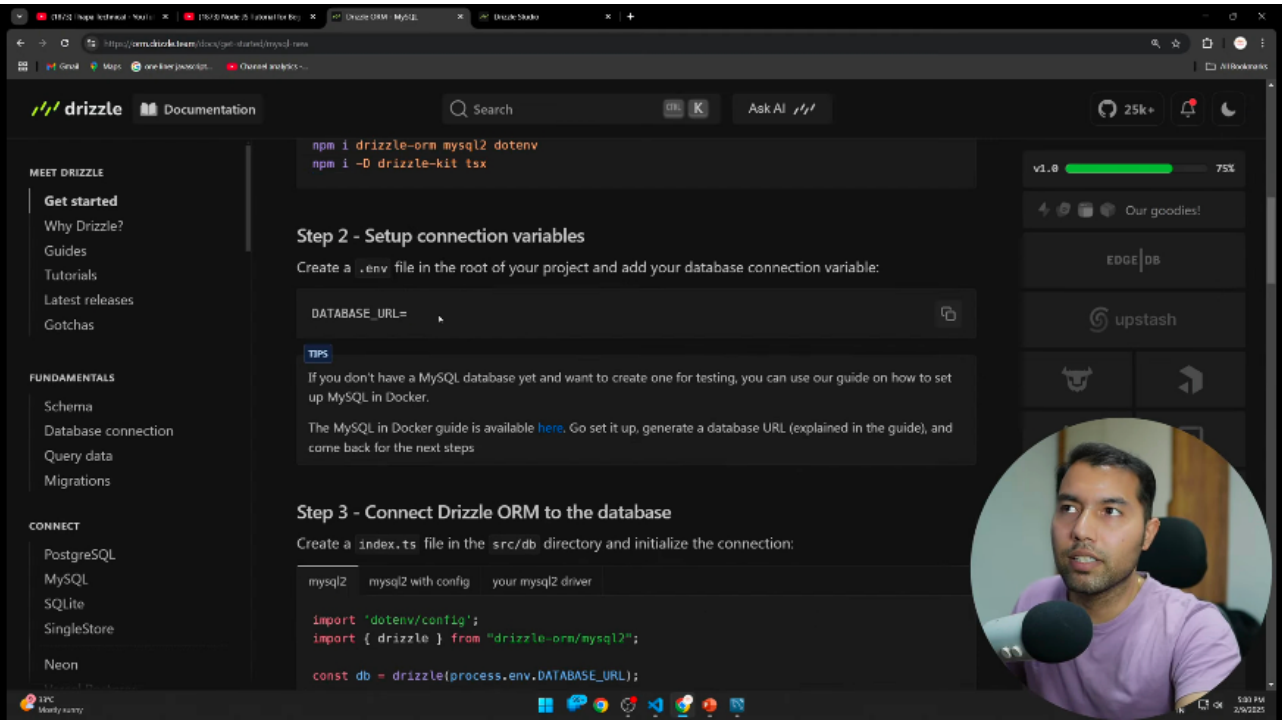

drizzle_mysql - Visual Studio Code
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
pwsh
F:\classes\drizzle_mysql
❯ npm init -y

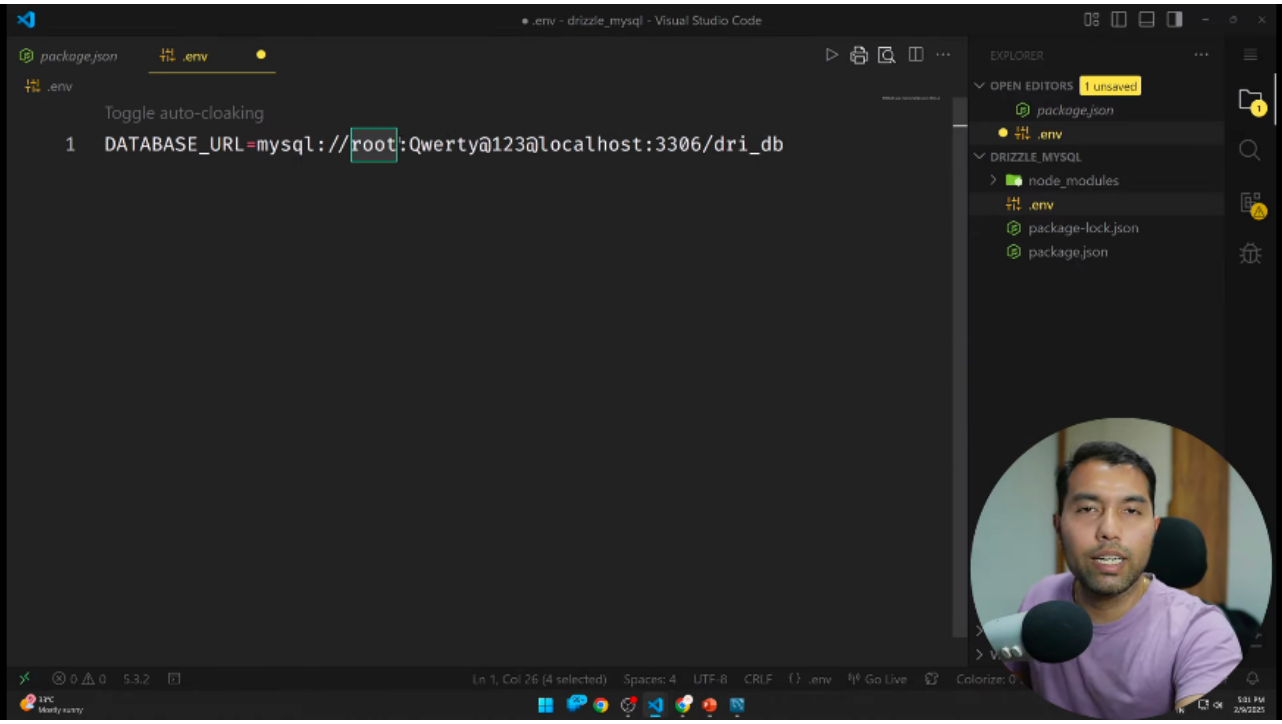

drizzle
Documentation
Search
Ask AI
25k+
MEET DRIZZLE
Get started
Why Drizzle?
Guides
Tutorials
Latest releases
Gotchas
FUNDAMENTALS
Schema
Database connection
Query data
Migrations
CONNECT
PostgreSQL
MySQL
SQLite
SingleStore
Neon
Basic file structure
This is the basic file structure of the project. In the src/db directory, we have table definition in schema.ts. In drizzle folder there are sql migration file and snapshots.
<project root>
drizzle
src
db
schema.ts
index.ts
.env
drizzle.config.ts
package.json
tsconfig.json
Step 1 - Install mysql2 package
npm
yarn
pnpm
bun
npm i drizzle-orm mysql2 dotenv
npm i -D drizzle-kit tsx
Step 2 - Setup connection variables
Create a .env file in the root of your project and add your database connection variable:
v1.0
75%
Our goodies!
EDGEDB
upstash

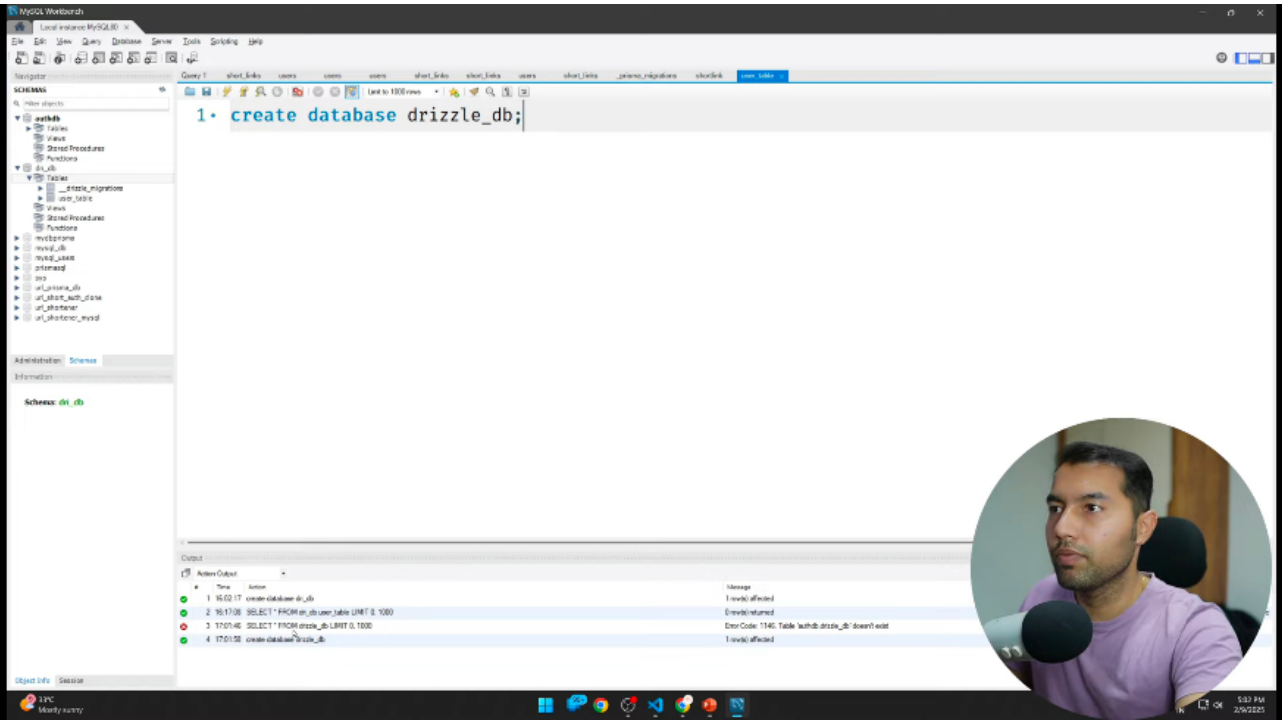

drizzle_mysql - Visual Studio Code
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
package.json
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
node
F:\classes\drizzle_mysql is v1.0.0 via v23.2.0
❯ npm i drizzle-orm mysql2
(node:4924) ExperimentalWarning: CommonJS module C:\Users\Thapa\AppData\Roaming\npm\node_modules\npm\node_modules\debug\src\node.js is loading ES Module C:\Users\Thapa\AppData\Roaming\npm\node_modules\npm\node_modules\supports-color\index.js using require().
Support for loading ES Module in require() is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
Go Live

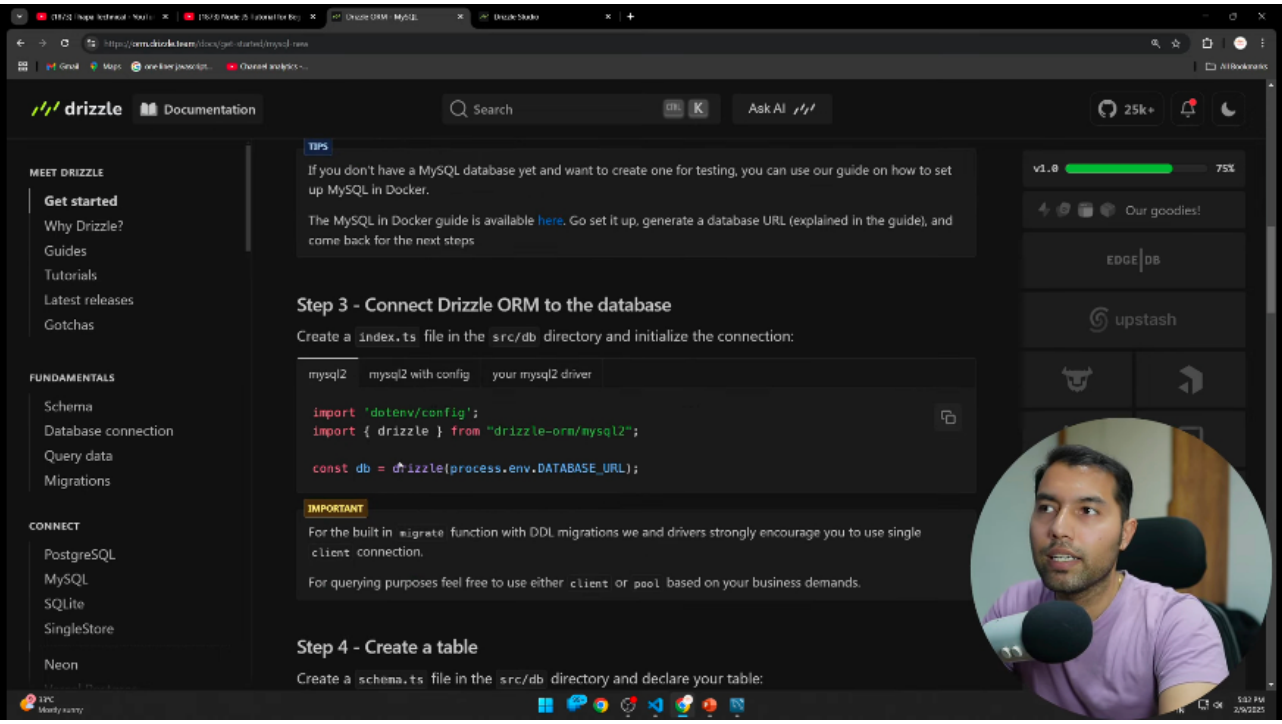

package.json - drizzle_mysql - Visual Studio Code
package.json
package.json > description
{
"name": "drizzle_mysql",
"version": "1.0.0",
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
pwsh
F:\classes\drizzle_mysql is v1.0.0 via v23.2.0
npm i -D drizzle-kit
EXPLORER
OPEN EDITORS
package.json
DRIZZLE_MYSQL
node_modules
package-lock.json
package.json
Ln 11, Col 4 (82 selected)
Spaces: 2
UTF-8
LF
Json
Go Live
Colorize: 0

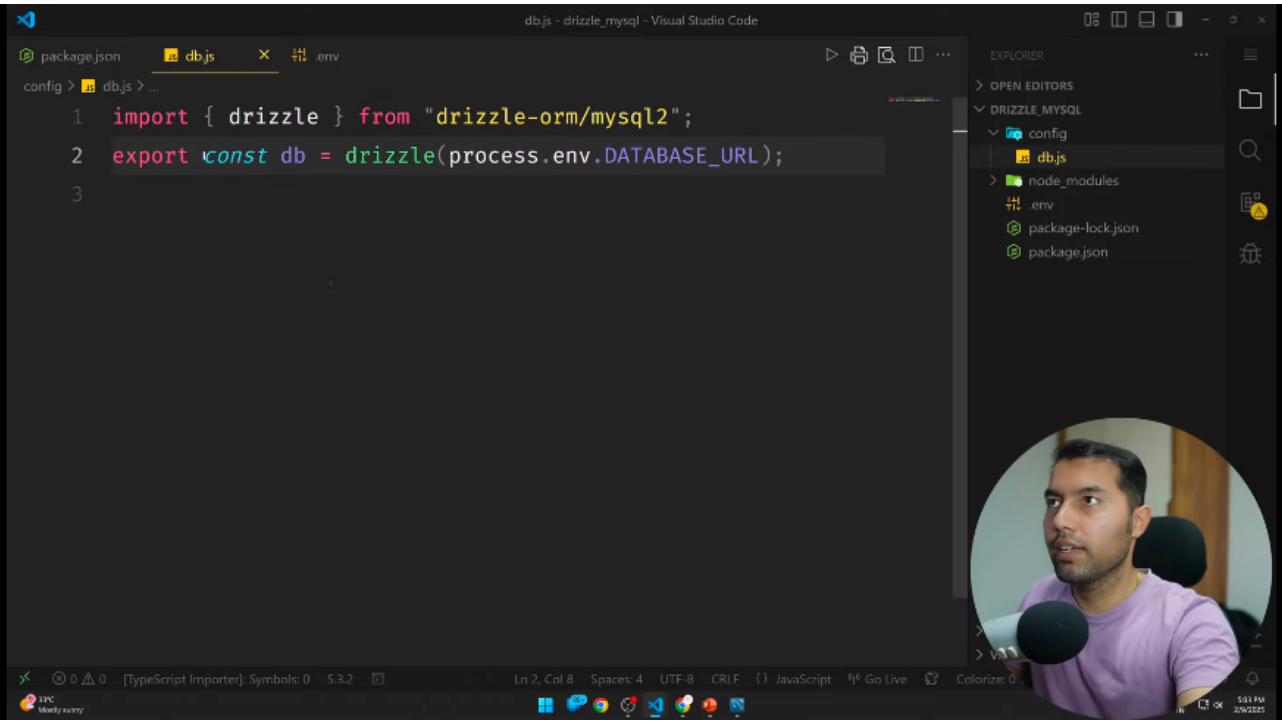

package.json - drizzle_mysql - Visual Studio Code
package.json
package.json > { } devDependencies > drizzle-kit
{
"name": "drizzle_mysql",
"version": "1.0.0",
"main": "index.js",
"scripts": {
"test": "echo \"Error: no test specified\" && exit 1"
},
"keywords": [],
"author": "",
"license": "ISC",
"description": "",
"dependencies": {
"drizzle-orm": "^0.39.2",
"mysql2": "^3.12.0"
},
"devDependencies": {
"drizzle-kit": "^0.30.4"
}
}
EXPLORER
OPEN EDITORS
package.json
DRIZZLE_MYSQL
node_modules
package-lock.json
package.json
Ln 17, Col 29
Spaces: 2
UTF-8
LF
{} Json
Go Live
Colorize: 0

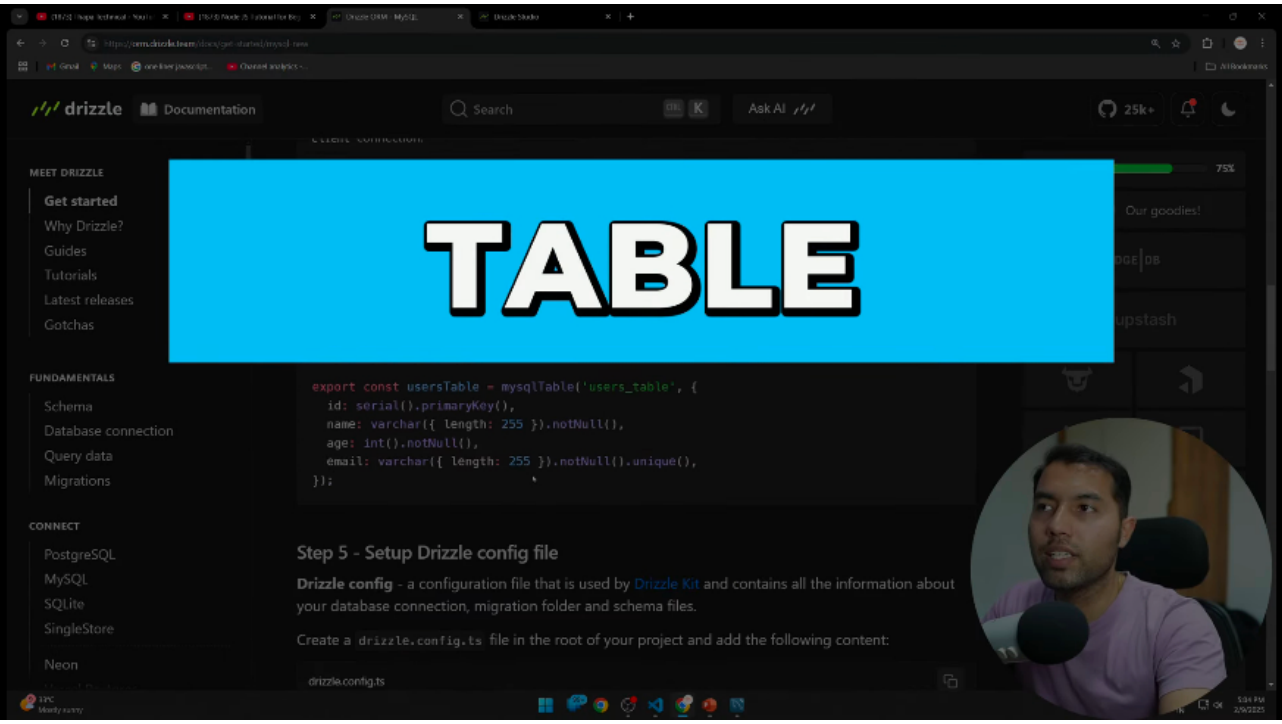

drizzle
Documentation
Search
Ask AI
25k+
MEET DRIZZLE
Get started
Why Drizzle?
Guides
Tutorials
Latest releases
Gotchas
FUNDAMENTALS
Schema
Database connection
Query data
Migrations
CONNECT
PostgreSQL
MySQL
SQLite
SingleStore
Neon
npm i drizzle-orm mysql2 dotenv
npm i -D drizzle-kit tsx
Step 2 - Setup connection variables
Create a .env file in the root of your project and add your database connection variable:
DATABASE_URL=
TIPS
If you don't have a MySQL database yet and want to create one for testing, you can use our guide on how to set up MySQL in Docker.
The MySQL in Docker guide is available here. Go set it up, generate a database URL (explained in the guide), and come back for the next steps
Step 3 - Connect Drizzle ORM to the database
Create a index.ts file in the src/db directory and initialize the connection:
mysql2
mysql2 with config
your mysql2 driver
import 'dotenv/config';
import { drizzle } from "drizzle-orm/mysql2";
const db = drizzle(process.env.DATABASE_URL);
v1.0
75%
Our goodies!
EDGEDB
upstash

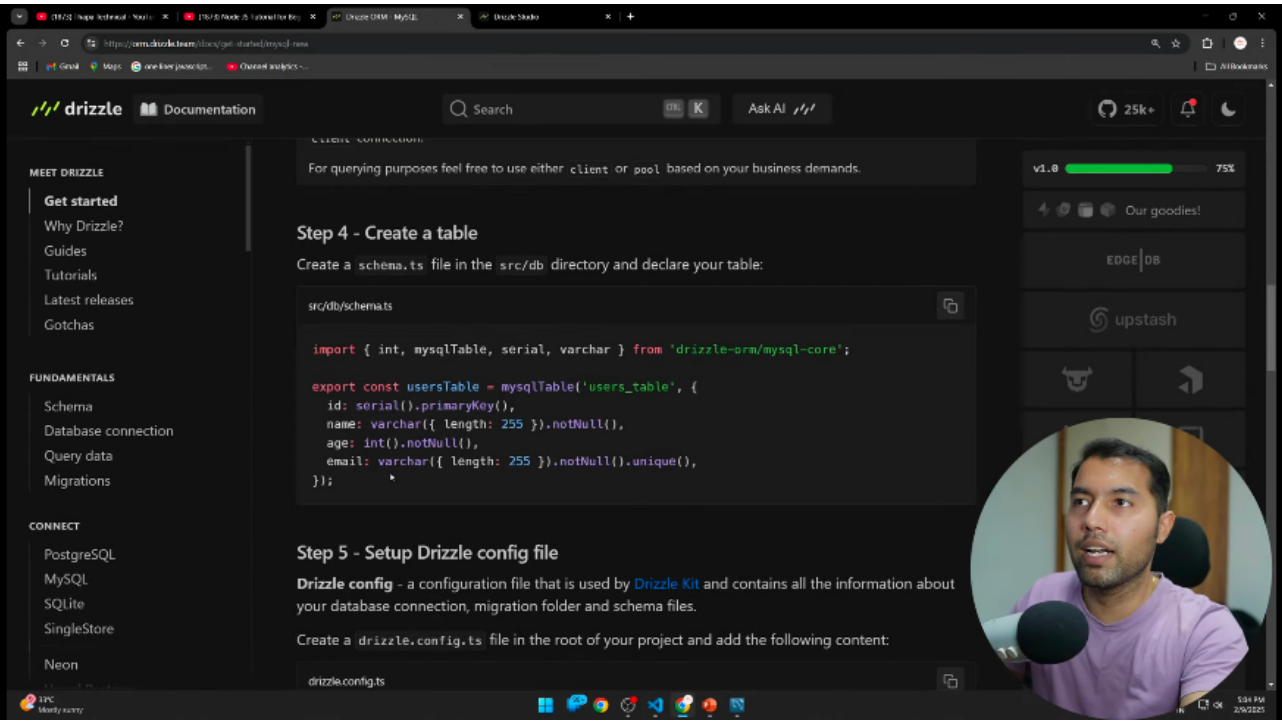

● .env - drizzle_mysql - Visual Studio Code
package.json
.env
.env
Toggle auto-cloaking
1 DATABASE_URL=mysql://root:Qwerty@123@localhost:3306/dri_db
EXPLORER
OPEN EDITORS 1 unsaved
package.json
.env
DRIZZLE_MYSQL
node_modules
.env
package-lock.json
package.json
Ln 1, Col 26 (4 selected) Spaces: 4 UTF-8 CRLF .env Go Live Colorize: 0

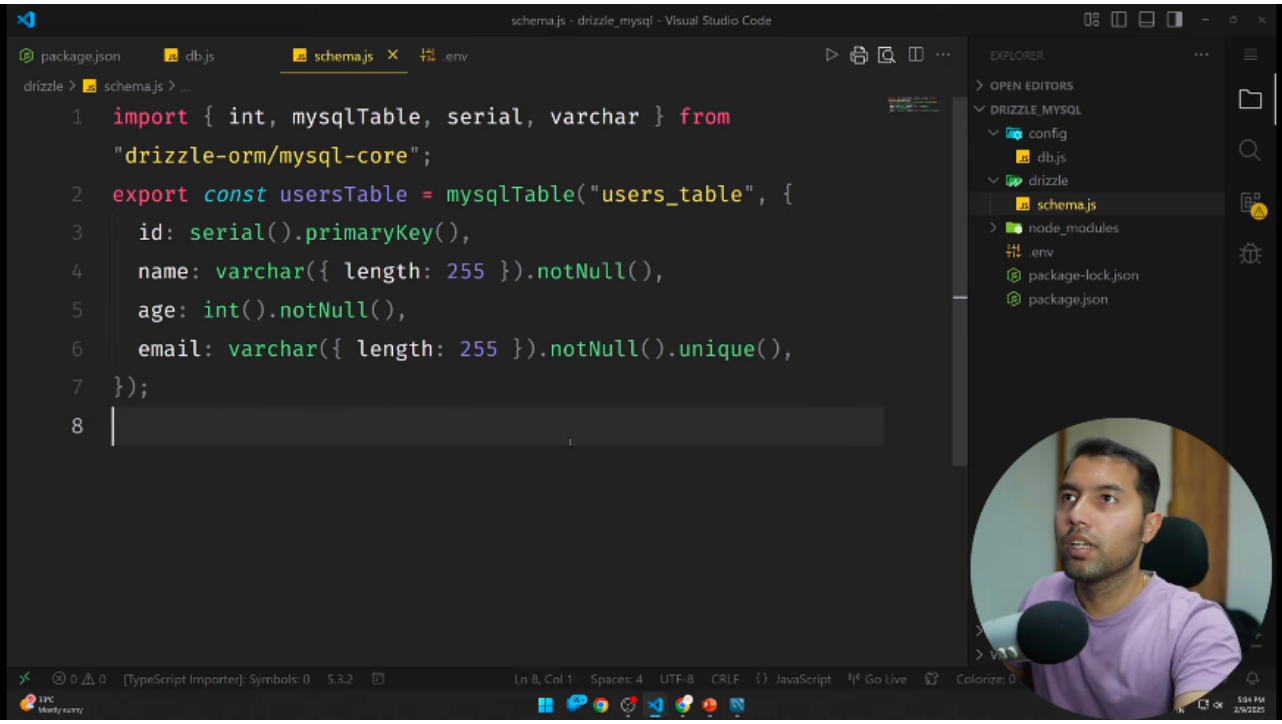

MySQL Workbench
create database drizzle_db;

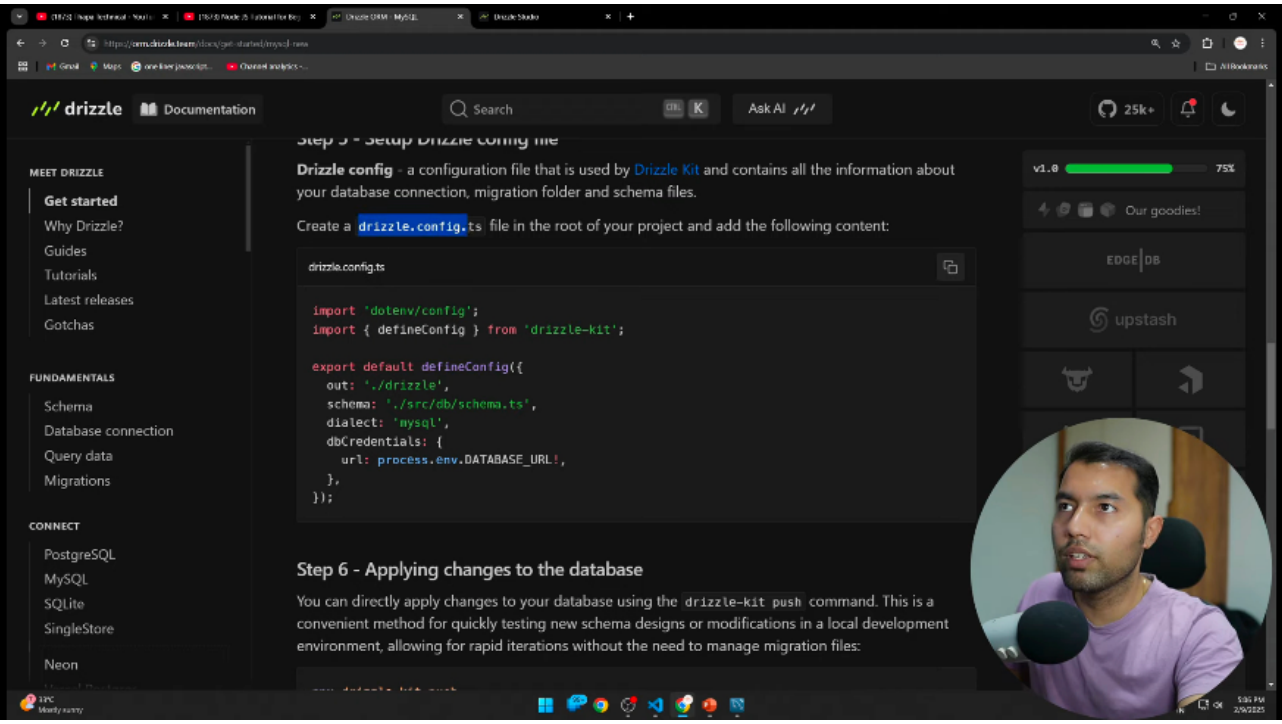

drizzle
Documentation
Search
Ask AI
25k+
MEET DRIZZLE
Get started
Why Drizzle?
Guides
Tutorials
Latest releases
Gotchas
FUNDAMENTALS
Schema
Database connection
Query data
Migrations
CONNECT
PostgreSQL
MySQL
SQLite
SingleStore
Neon
TIPS
If you don't have a MySQL database yet and want to create one for testing, you can use our guide on how to set up MySQL in Docker.
The MySQL in Docker guide is available here. Go set it up, generate a database URL (explained in the guide), and come back for the next steps
Step 3 - Connect Drizzle ORM to the database
Create a index.ts file in the src/db directory and initialize the connection:
mysql2
mysql2 with config
your mysql2 driver
import 'dotenv/config';
import { drizzle } from "drizzle-orm/mysql2";
const db = drizzle(process.env.DATABASE_URL);
IMPORTANT
For the built in migrate function with DDL migrations we and drivers strongly encourage you to use single client connection.
For querying purposes feel free to use either client or pool based on your business demands.
Step 4 - Create a table
Create a schema.ts file in the src/db directory and declare your table:
v1.0
75%
Our goodies!
EDGE DB
upstash

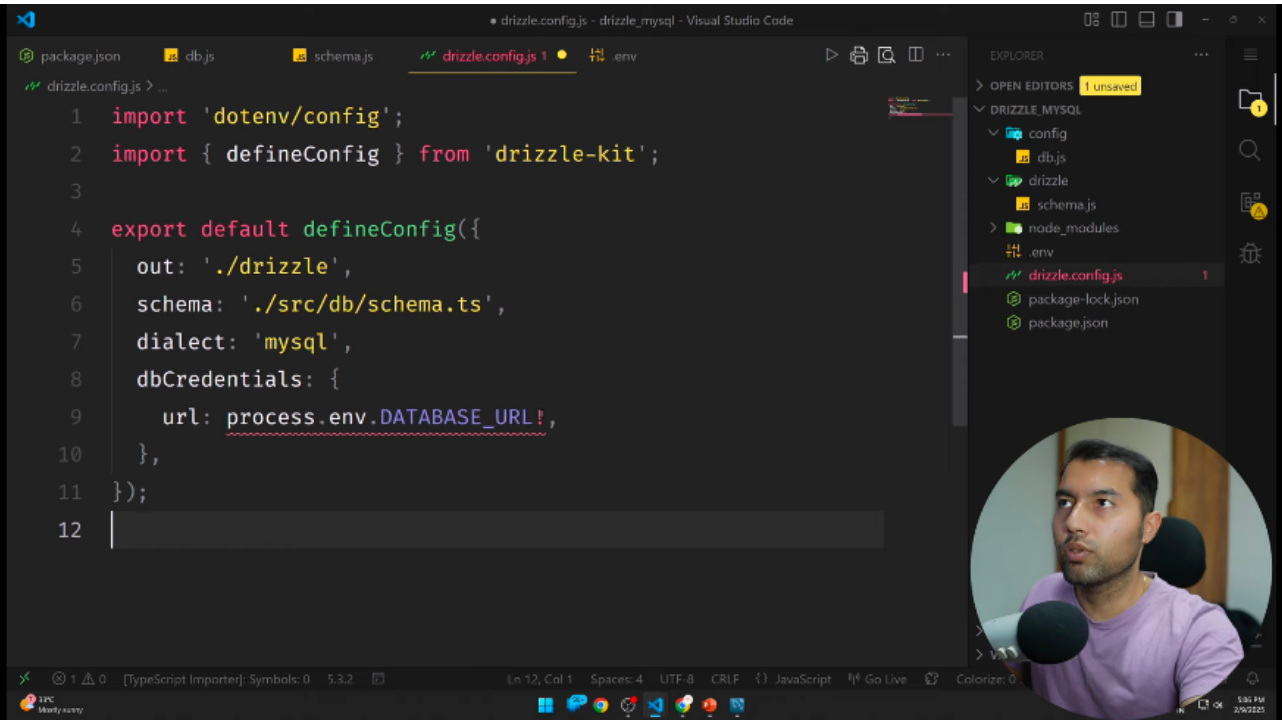

db.js - drizzle_mysql - Visual Studio Code
package.json
db.js
.env
config > db.js > ...
1 import { drizzle } from "drizzle-orm/mysql2";
2 export const db = drizzle(process.env.DATABASE_URL);
3
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
node_modules
.env
package-lock.json
package.json
Ln 2, Col 8
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
Colorize: 0

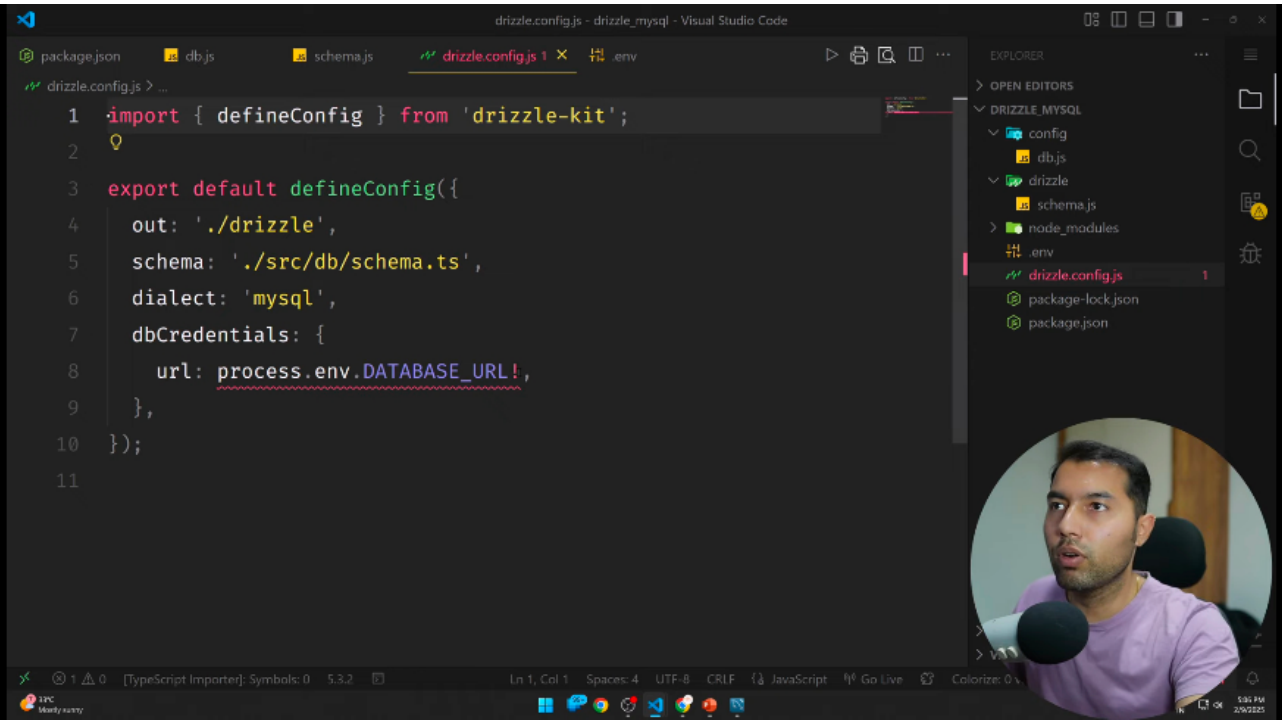

drizzle
Documentation
Search
Ask AI
25k+
MEET DRIZZLE
Get started
Why Drizzle?
Guides
Tutorials
Latest releases
Gotchas
FUNDAMENTALS
Schema
Database connection
Query data
Migrations
CONNECT
PostgreSQL
MySQL
SQLite
SingleStore
Neon
TABLE
export const usersTable = mysqlTable('users_table', {
id: serial().primaryKey(),
name: varchar({ length: 255 }).notNull(),
age: int().notNull(),
email: varchar({ length: 255 }).notNull().unique(),
});
Step 5 - Setup Drizzle config file
Drizzle config - a configuration file that is used by Drizzle Kit and contains all the information about your database connection, migration folder and schema files.
Create a drizzle.config.ts file in the root of your project and add the following content:
drizzle.config.ts

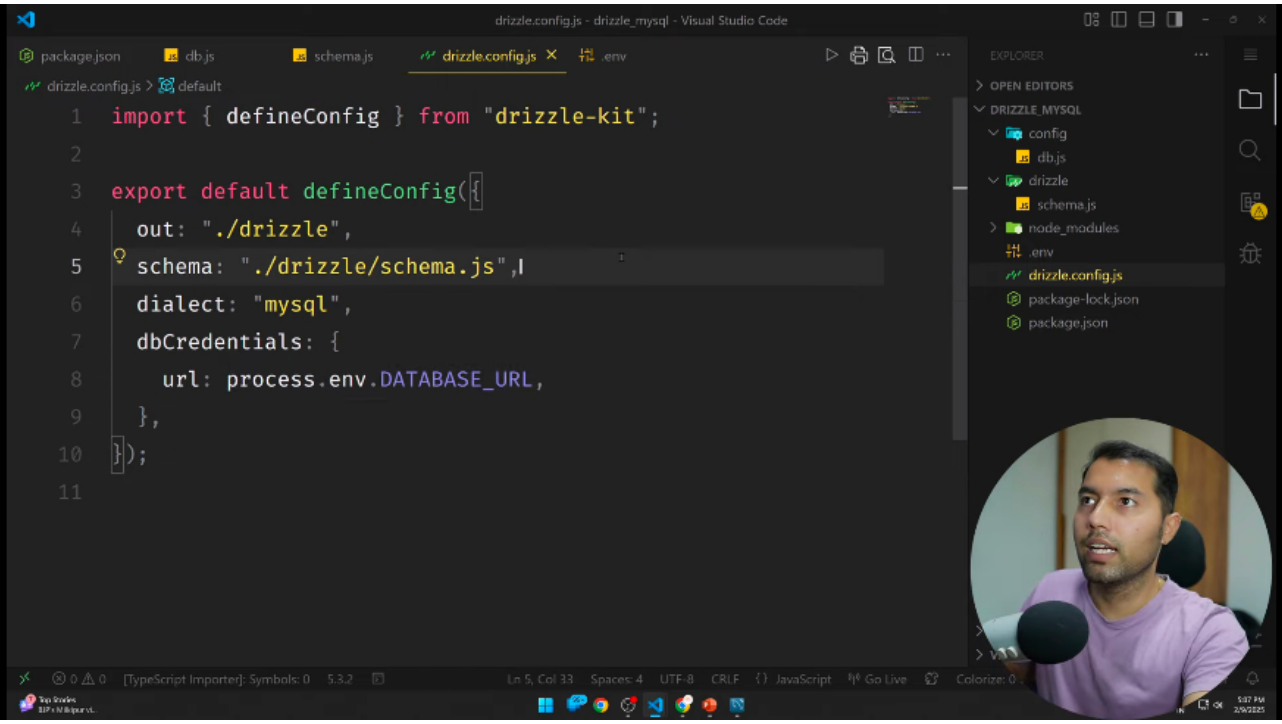

drizzle
Documentation
Search
Ask AI
25k+
MEET DRIZZLE
Get started
Why Drizzle?
Guides
Tutorials
Latest releases
Gotchas
FUNDAMENTALS
Schema
Database connection
Query data
Migrations
CONNECT
PostgreSQL
MySQL
SQLite
SingleStore
Neon
For querying purposes feel free to use either client or pool based on your business demands.
Step 4 - Create a table
Create a schema.ts file in the src/db directory and declare your table:
src/db/schema.ts
import { int, mysqlTable, serial, varchar } from 'drizzle-orm/mysql-core';
export const usersTable = mysqlTable('users_table', {
id: serial().primaryKey(),
name: varchar({ length: 255 }).notNull(),
age: int().notNull(),
email: varchar({ length: 255 }).notNull().unique(),
});
Step 5 - Setup Drizzle config file
Drizzle config - a configuration file that is used by Drizzle Kit and contains all the information about your database connection, migration folder and schema files.
Create a drizzle.config.ts file in the root of your project and add the following content:
drizzle.config.ts
v1.0
75%
Our goodies!
EDGE DB
upstash

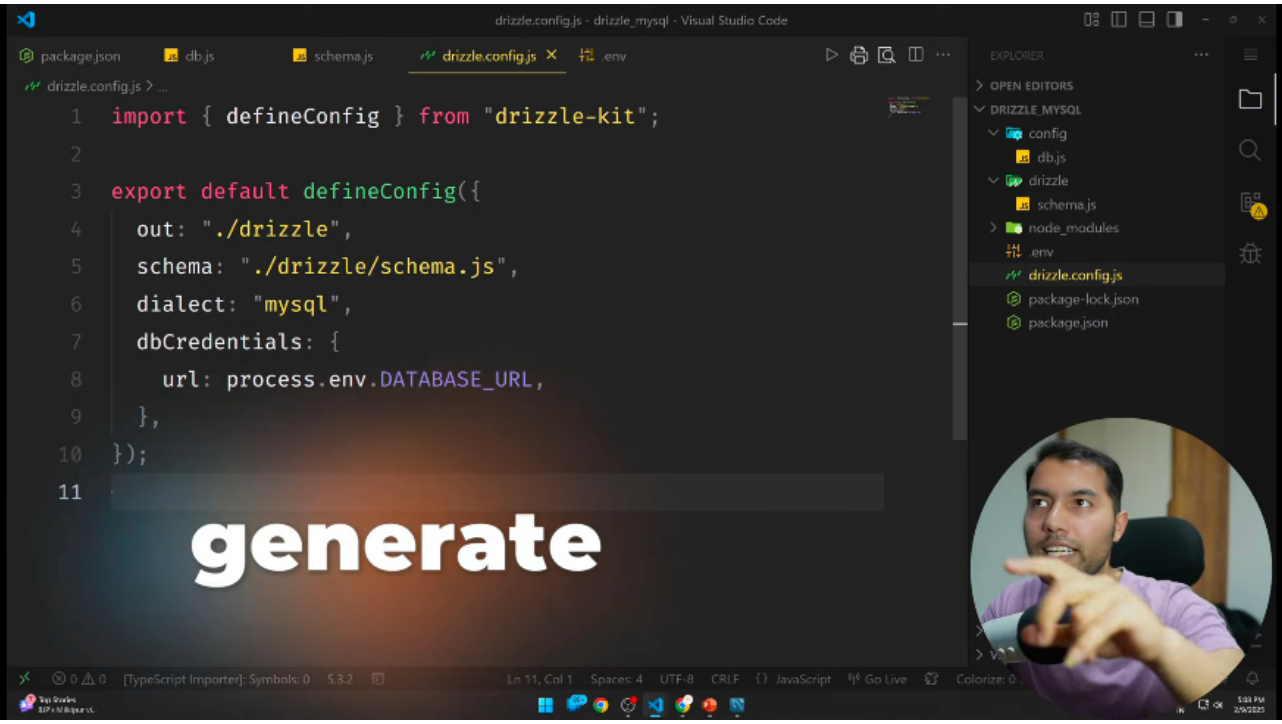

schema.js - drizzle_mysql - Visual Studio Code
package.json
db.js
schema.js
.env
drizzle > schema.js > ...
1 import { int, mysqlTable, serial, varchar } from "drizzle-orm/mysql-core";
2 export const usersTable = mysqlTable("users_table", {
3 id: serial().primaryKey(),
4 name: varchar({ length: 255 }).notNull(),
5 age: int().notNull(),
6 email: varchar({ length: 255 }).notNull().unique(),
7 });
8
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
schema.js
node_modules
.env
package-lock.json
package.json
Ln 8, Col 1
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
Colorize: 0

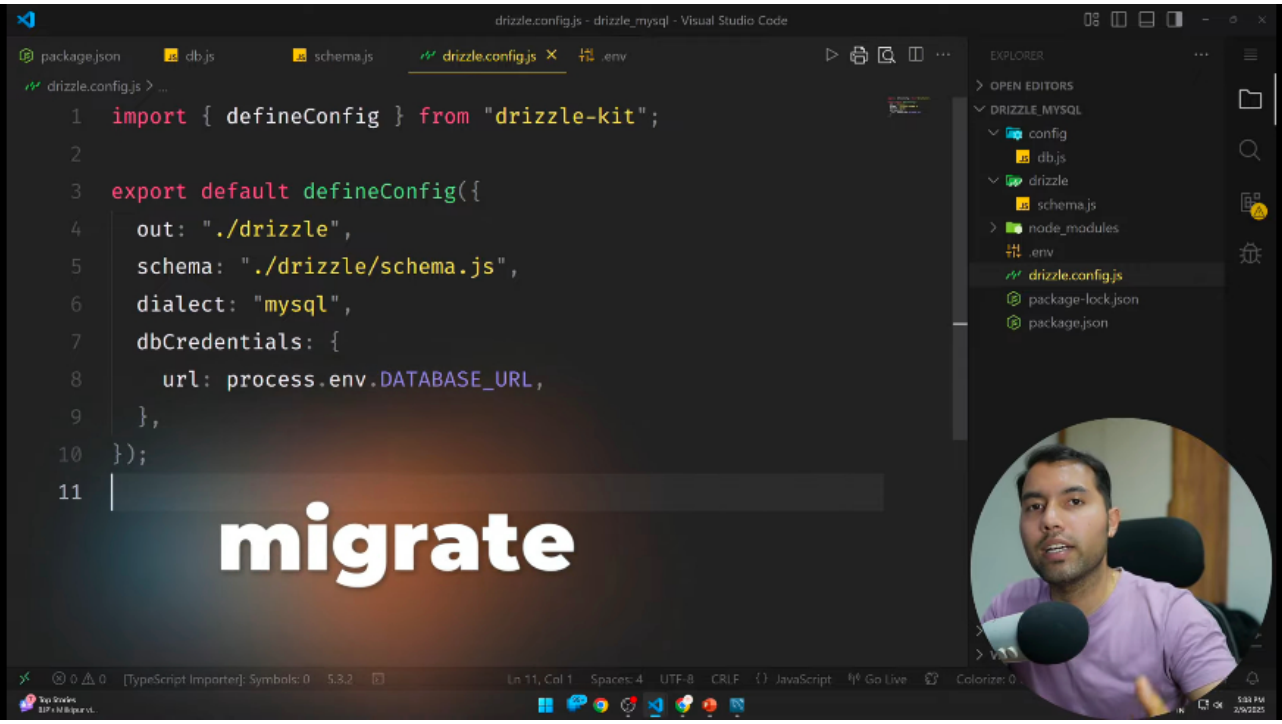

drizzle
Documentation
Search
Ask AI
25k+
MEET DRIZZLE
Get started
Why Drizzle?
Guides
Tutorials
Latest releases
Gotchas
FUNDAMENTALS
Schema
Database connection
Query data
Migrations
CONNECT
PostgreSQL
MySQL
SQLite
SingleStore
Neon
Drizzle config - a configuration file that is used by Drizzle Kit and contains all the information about your database connection, migration folder and schema files.
Create a drizzle.config.ts file in the root of your project and add the following content:
drizzle.config.ts
import 'dotenv/config';
import { defineConfig } from 'drizzle-kit';
export default defineConfig({
out: './drizzle',
schema: './src/db/schema.ts',
dialect: 'mysql',
dbCredentials: {
url: process.env.DATABASE_URL!,
},
});
Step 6 - Applying changes to the database
You can directly apply changes to your database using the drizzle-kit push command. This is a convenient method for quickly testing new schema designs or modifications in a local development environment, allowing for rapid iterations without the need to manage migration files:
v1.0
75%
Our goodies!
EDGE DB
upstash

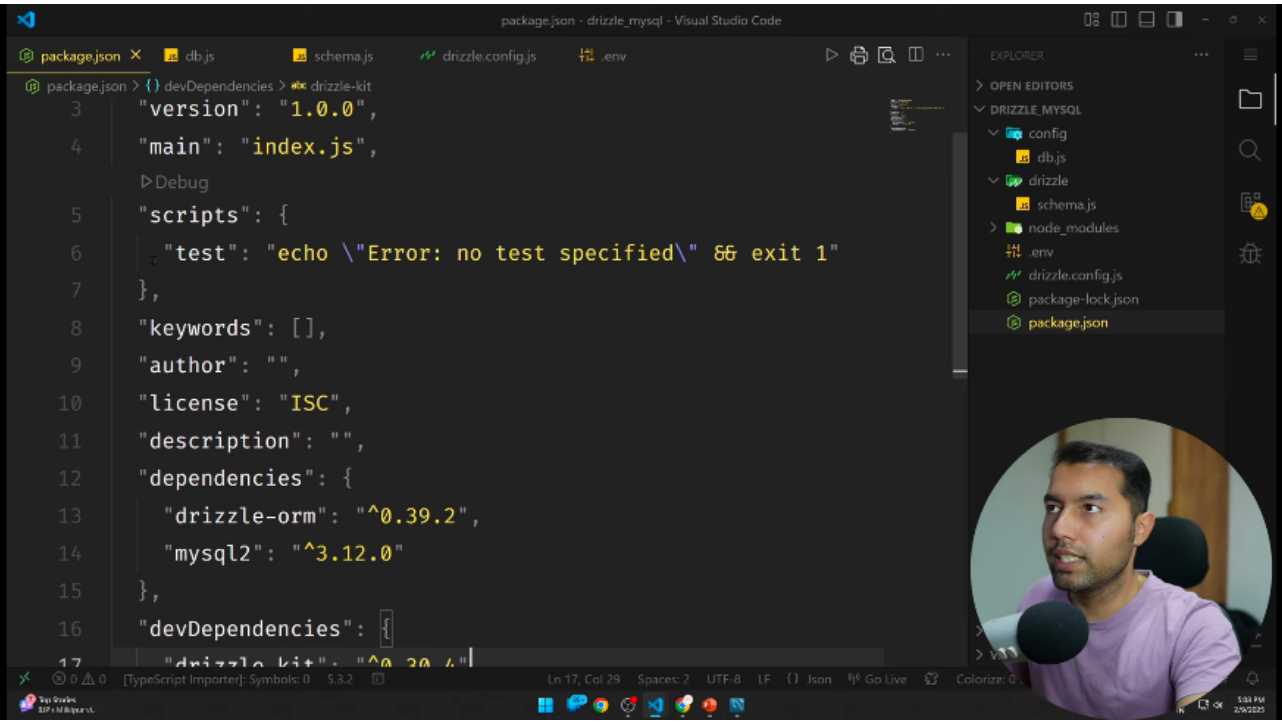

drizzle.config.js - drizzle_mysql - Visual Studio Code
package.json
db.js
schema.js
drizzle.config.js 1
.env
drizzle.config.js > ...
1 import 'dotenv/config';
2 import { defineConfig } from 'drizzle-kit';
3
4 export default defineConfig({
5 out: './drizzle',
6 schema: './src/db/schema.ts',
7 dialect: 'mysql',
8 dbCredentials: {
9 url: process.env.DATABASE_URL!,
10 },
11 });
12
EXPLORER
OPEN EDITORS 1 unsaved
DRIZZLE_MYSQL
config
db.js
drizzle
schema.js
node_modules
.env
drizzle.config.js 1
package-lock.json
package.json
Ln 12, Col 1
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
Colorize: 0

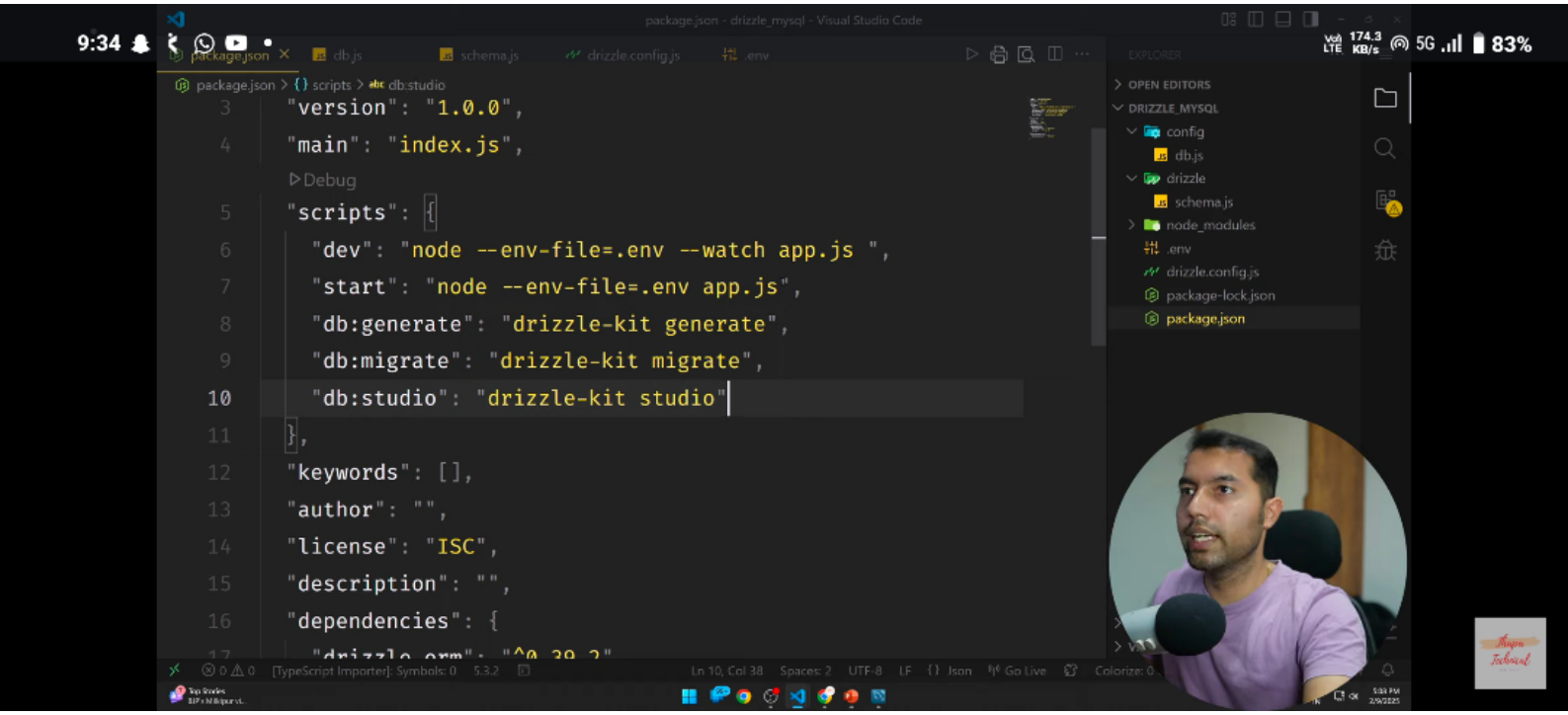

drizzle.config.js - drizzle_mysql - Visual Studio Code
package.json
db.js
schema.js
drizzle.config.js 1
.env
drizzle.config.js > ...
1 import { defineConfig } from 'drizzle-kit';
2
3 export default defineConfig({
4 out: './drizzle',
5 schema: './src/db/schema.ts',
6 dialect: 'mysql',
7 dbCredentials: {
8 url: process.env.DATABASE_URL!,
9 },
10 });
11
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
schema.js
node_modules
.env
drizzle.config.js 1
package-lock.json
package.json
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live

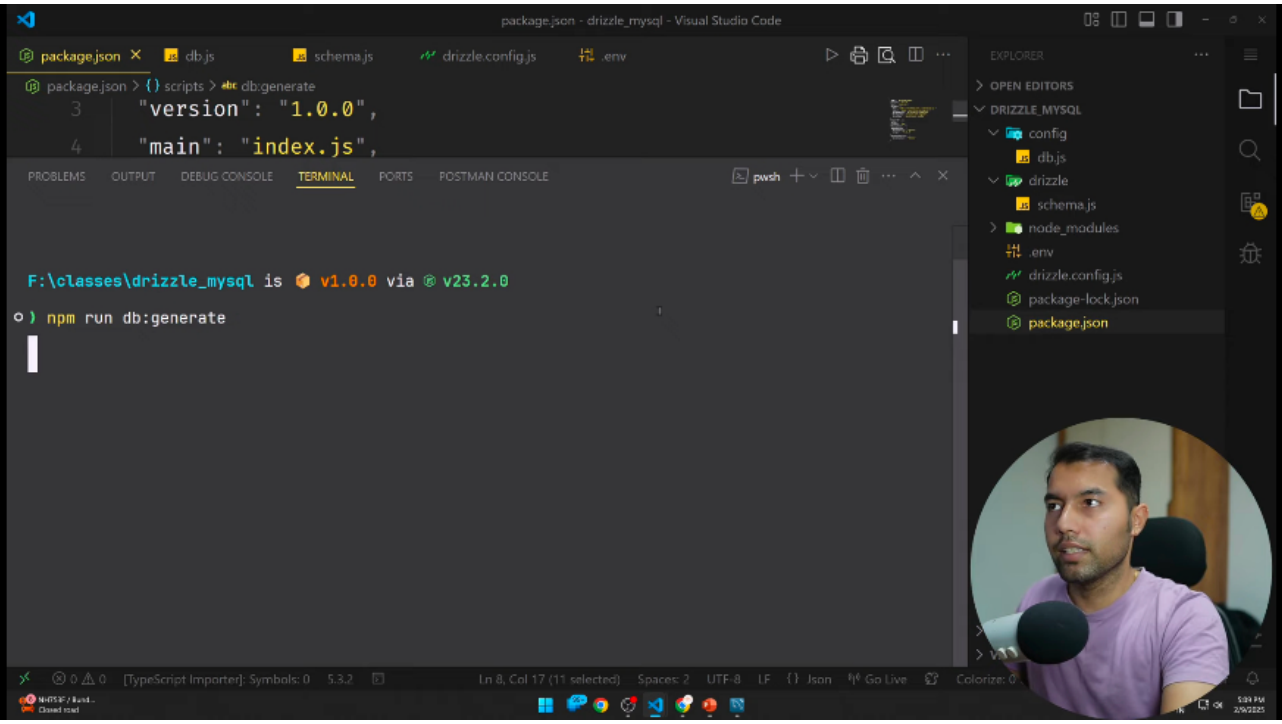

drizzle.config.js - drizzle_mysql - Visual Studio Code
package.json
db.js
schema.js
drizzle.config.js
.env
drizzle.config.js > default
import { defineConfig } from "drizzle-kit";
export default defineConfig({
out: "./drizzle",
schema: "./drizzle/schema.js",
dialect: "mysql",
dbCredentials: {
url: process.env.DATABASE_URL,
},
});
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
schema.js
node_modules
.env
drizzle.config.js
package-lock.json
package.json

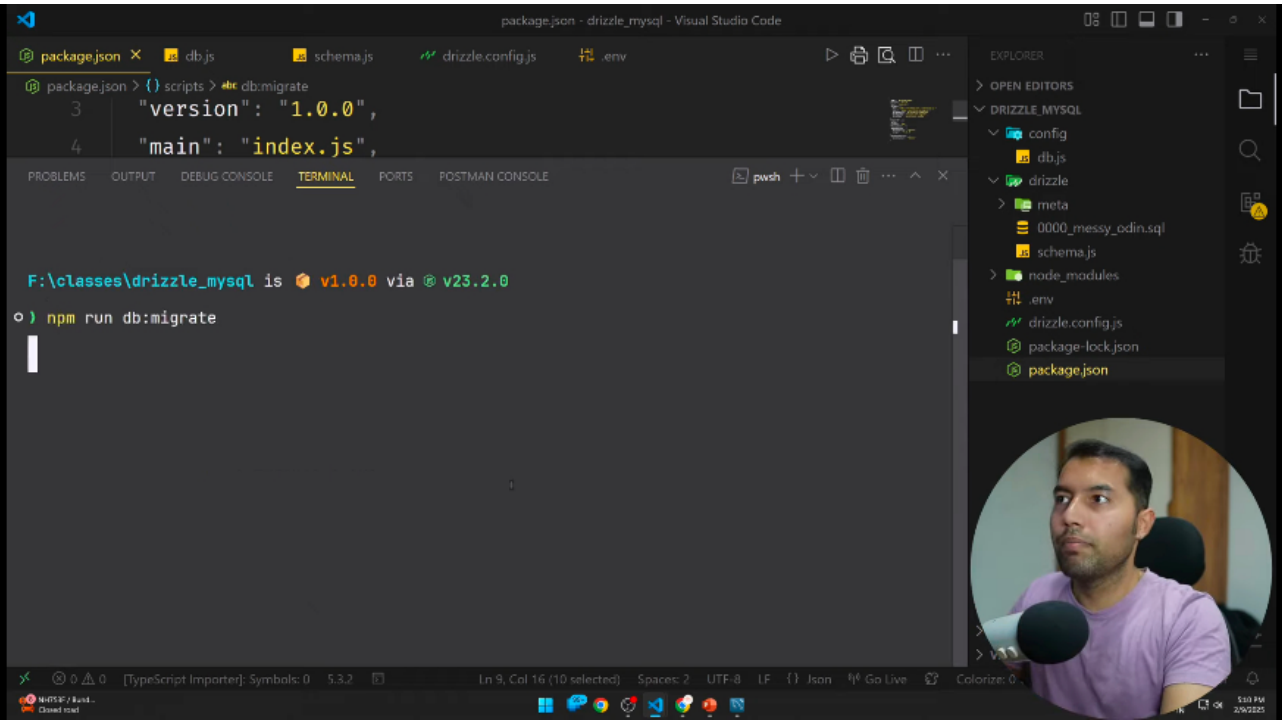

drizzle.config.js - drizzle_mysql - Visual Studio Code
package.json
db.js
schema.js
drizzle.config.js
.env
drizzle.config.js > ...
import { defineConfig } from "drizzle-kit";
export default defineConfig({
out: "./drizzle",
schema: "./drizzle/schema.js",
dialect: "mysql",
dbCredentials: {
url: process.env.DATABASE_URL,
},
});
generate
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
schema.js
node_modules
.env
drizzle.config.js
package-lock.json
package.json

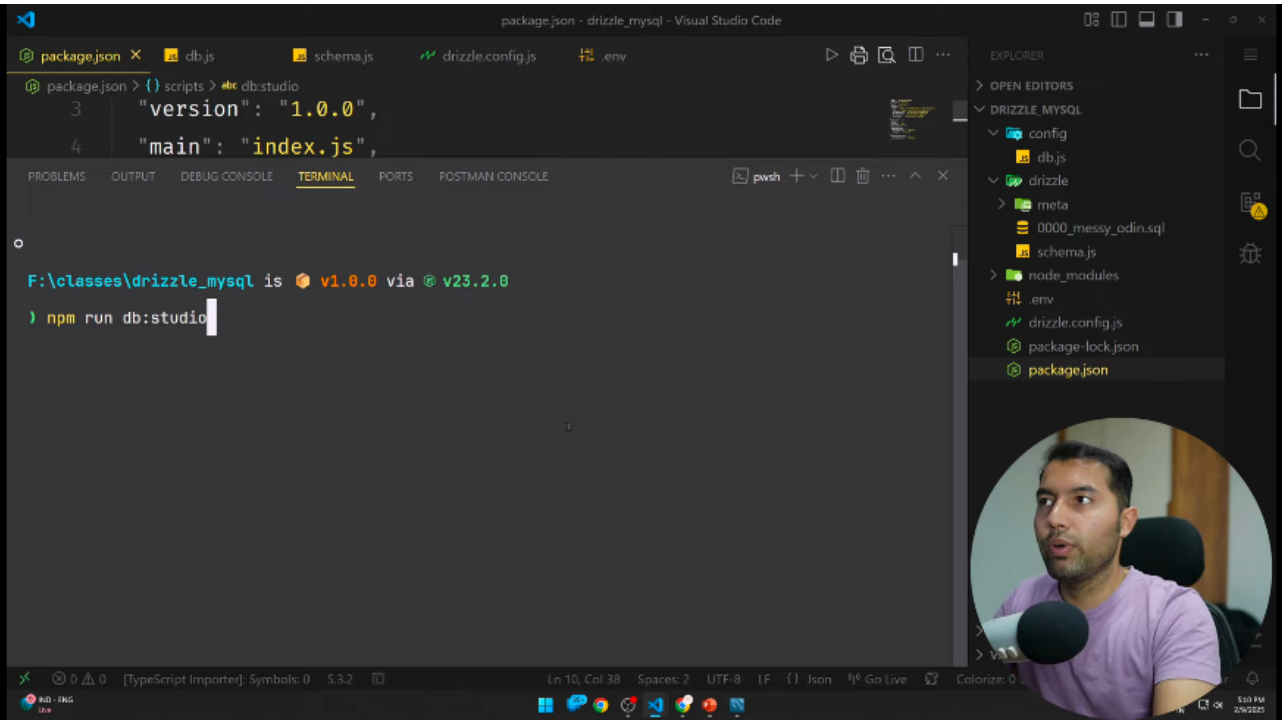

drizzle.config.js - drizzle_mysql - Visual Studio Code
package.json
db.js
schema.js
drizzle.config.js
.env
drizzle.config.js > ...
import { defineConfig } from "drizzle-kit";
export default defineConfig({
out: "./drizzle",
schema: "./drizzle/schema.js",
dialect: "mysql",
dbCredentials: {
url: process.env.DATABASE_URL,
},
});
migrate
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
schema.js
node_modules
.env
drizzle.config.js
package-lock.json
package.json

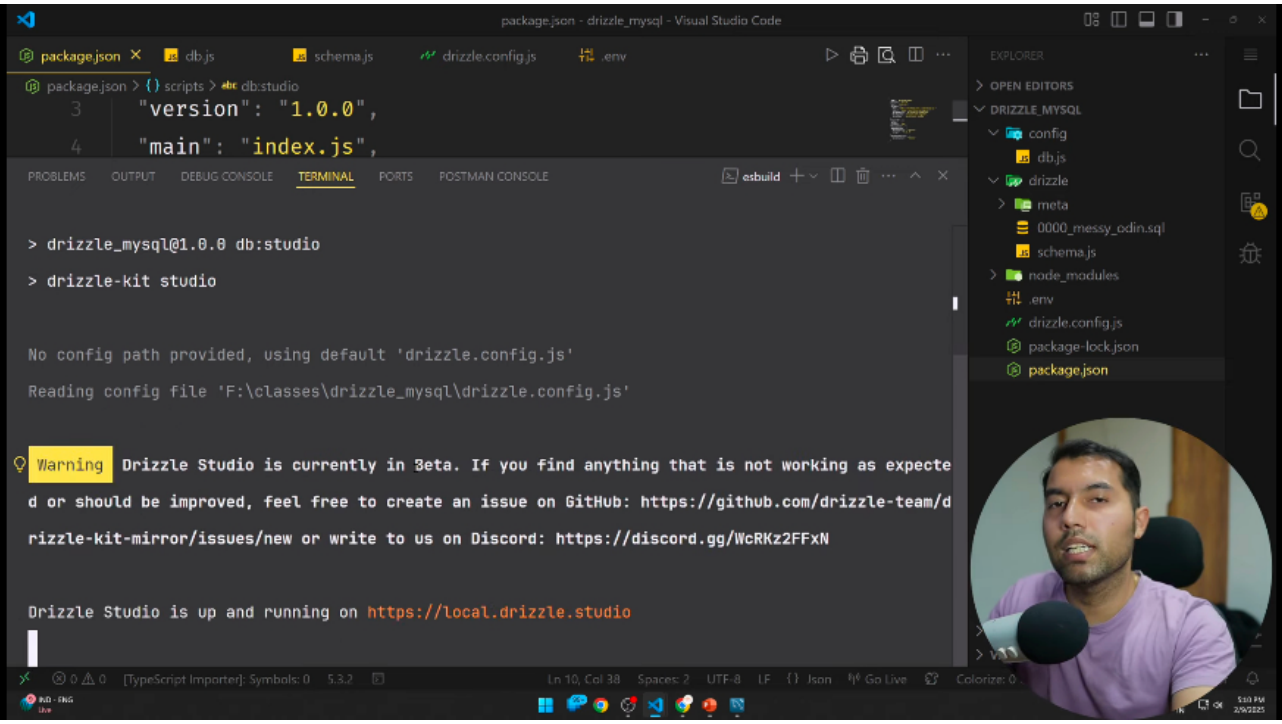

package.json - drizzle_mysql - Visual Studio Code
package.json
db.js
schema.js
drizzle.config.js
.env
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
schema.js
node_modules
.env
drizzle.config.js
package-lock.json
package.json
"version": "1.0.0",
"main": "index.js",
Debug
"scripts": {
"test": "echo \"Error: no test specified\" && exit 1"
},
"keywords": [],
"author": "",
"license": "ISC",
"description": "",
"dependencies": {
"drizzle-orm": "^0.39.2",
"mysql2": "^3.12.0"
},
"devDependencies": {

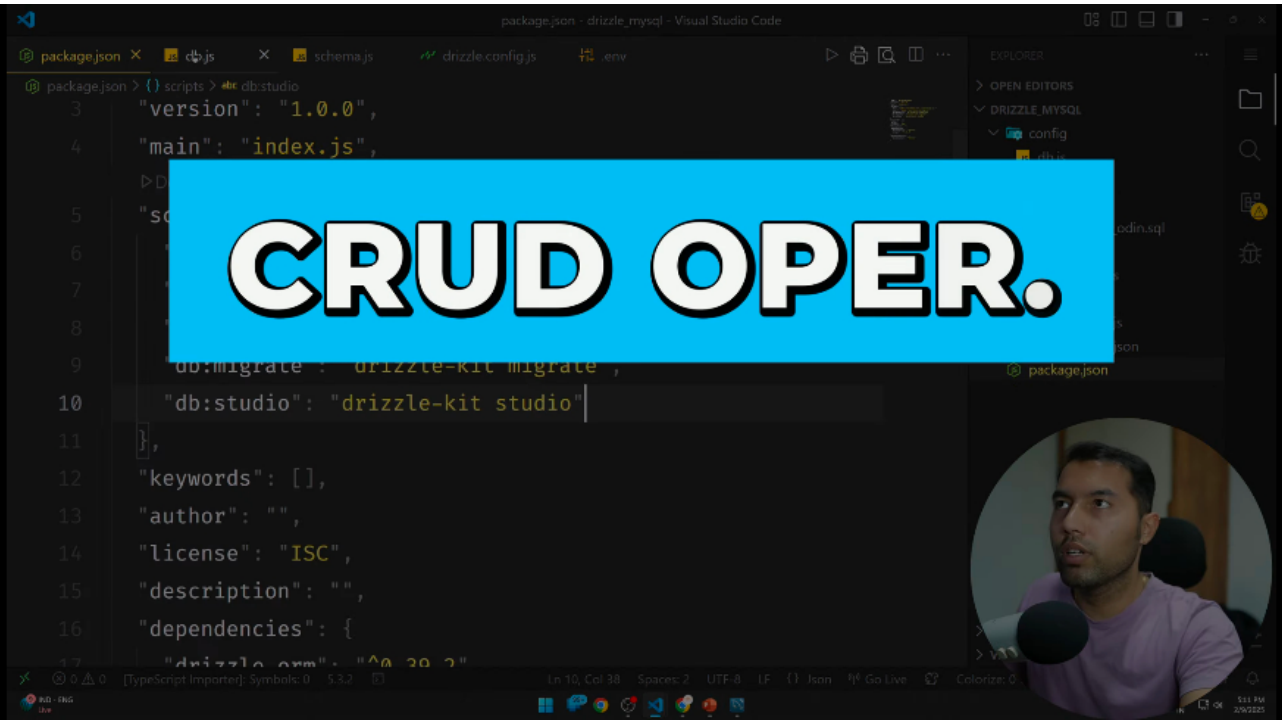

9:34
package.json - drizzle_mysql - Visual Studio Code
db.js
schema.js
drizzle.config.js
.env
package.json > { } scripts > db:studio
"version": "1.0.0",
"main": "index.js",
Debug
"scripts": {
"dev": "node --env-file=.env --watch app.js ",
"start": "node --env-file=.env app.js",
"db:generate": "drizzle-kit generate",
"db:migrate": "drizzle-kit migrate",
"db:studio": "drizzle-kit studio"
},
"keywords": [],
"author": "",
"license": "ISC",
"description": "",
"dependencies": {
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
schema.js
node_modules
.env
drizzle.config.js
package-lock.json
package.json
Ln 10, Col 38
Spaces: 2
UTF-8
LF
Json
Go Live
83%

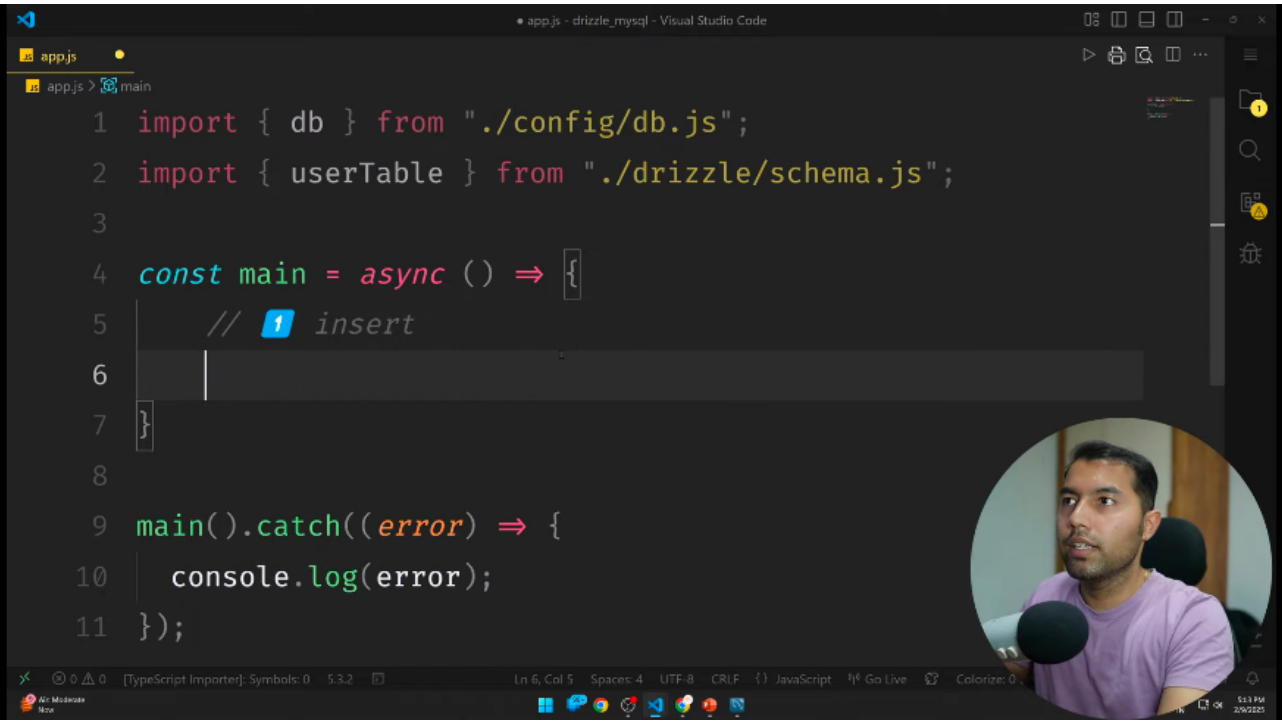

package.json - drizzle_mysql - Visual Studio Code
package.json
db.js
schema.js
drizzle.config.js
.env
"version": "1.0.0",
"main": "index.js",
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
F:\classes\drizzle_mysql is v1.0.0 via v23.2.0
npm run db:generate
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
schema.js
node_modules
.env
drizzle.config.js
package-lock.json
package.json

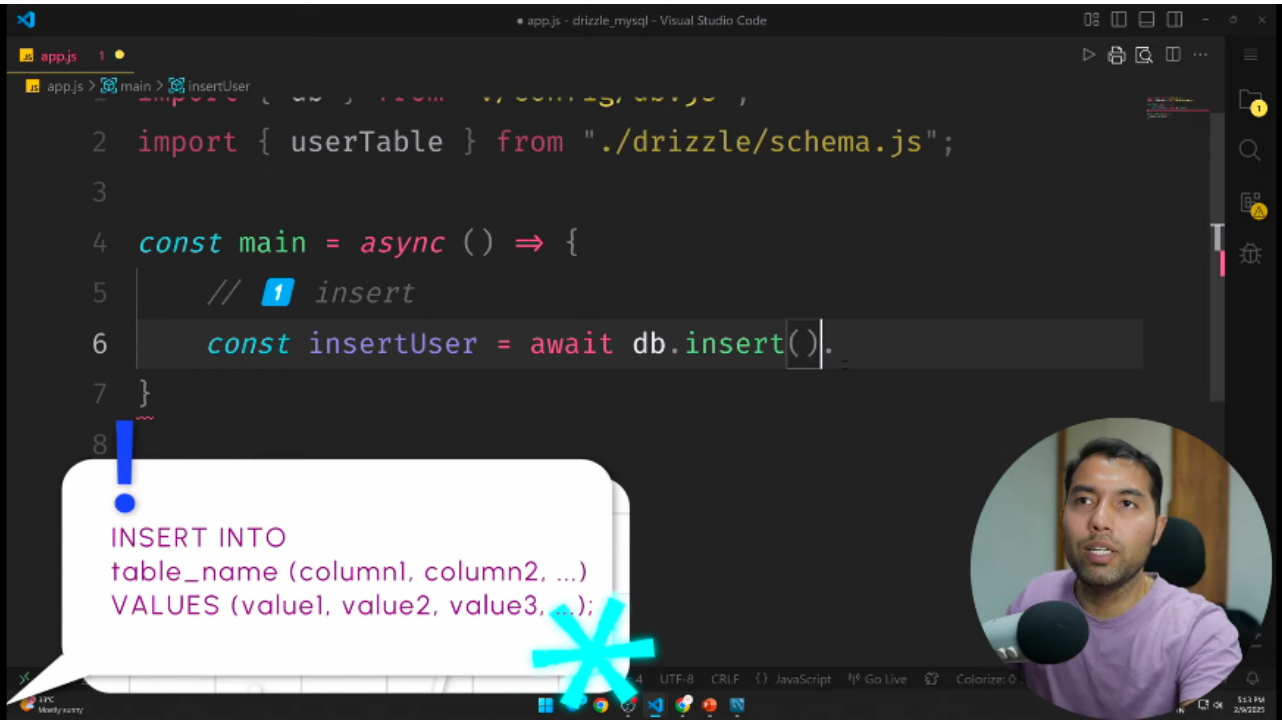

package.json - drizzle_mysql - Visual Studio Code
package.json
db.js
schema.js
drizzle.config.js
.env
"version": "1.0.0",
"main": "index.js",
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
F:\classes\drizzle_mysql is v1.0.0 via v23.2.0
npm run db:migrate
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
meta
0000_messy_odin.sql
schema.js
node_modules
.env
drizzle.config.js
package-lock.json
package.json

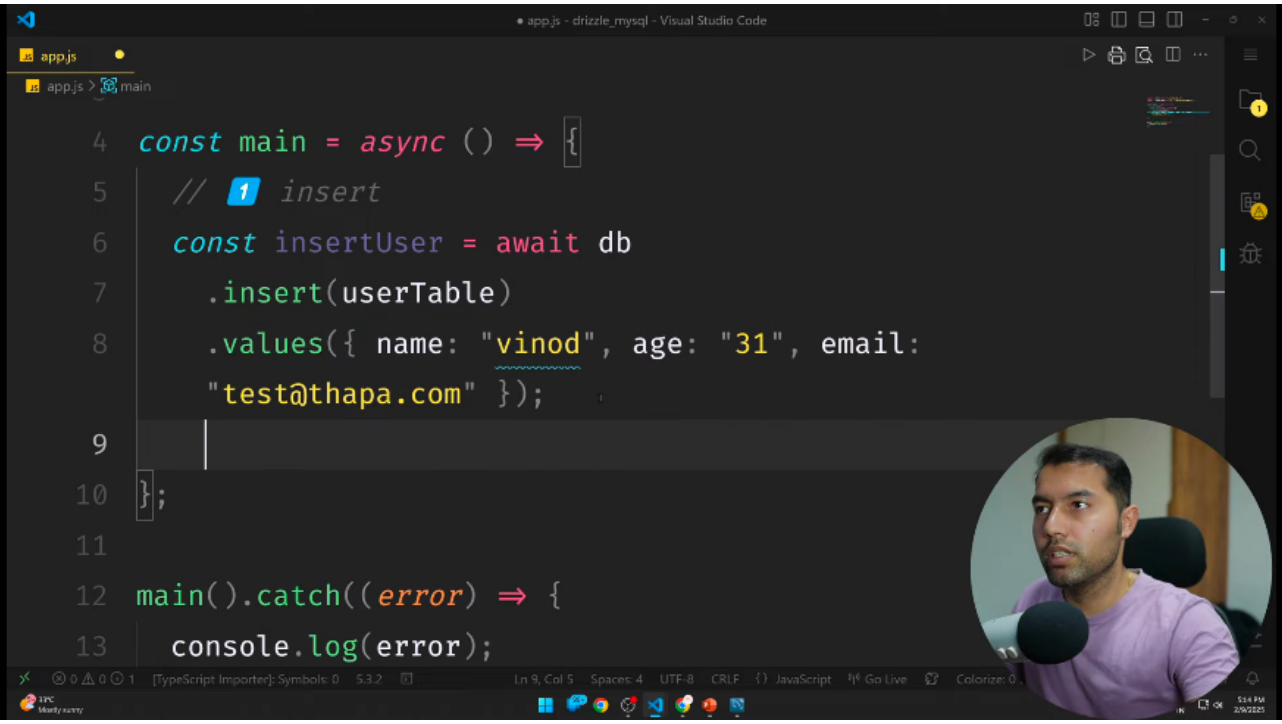

package.json - drizzle_mysql - Visual Studio Code
package.json
db.js
schema.js
drizzle.config.js
.env
"version": "1.0.0",
"main": "index.js",
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
F:\classes\drizzle_mysql is v1.0.0 via v23.2.0
npm run db:studio
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
meta
0000_messy_odin.sql
schema.js
node_modules
.env
drizzle.config.js
package-lock.json
package.json

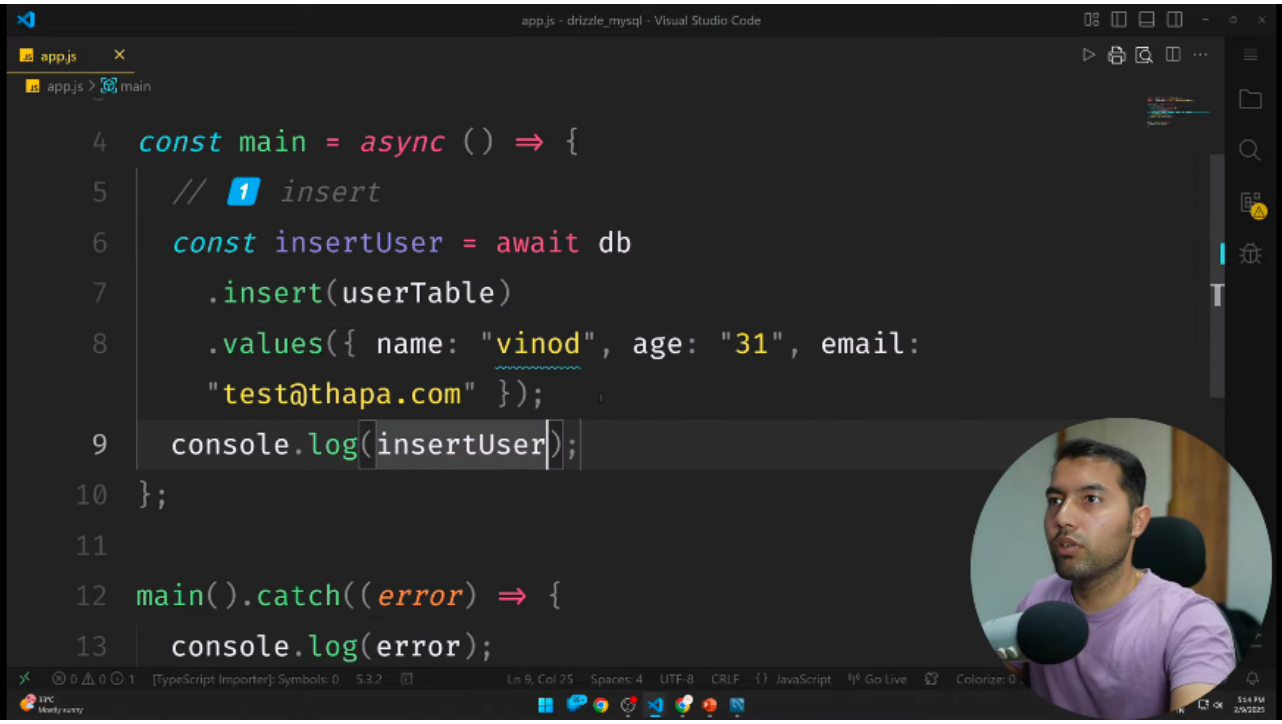

package.json - drizzle_mysql - Visual Studio Code
package.json
db.js
schema.js
drizzle.config.js
.env
"version": "1.0.0",
"main": "index.js",
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
esbuild
> drizzle_mysql@1.0.0 db:studio
> drizzle-kit studio
No config path provided, using default 'drizzle.config.js'
Reading config file 'F:\classes\drizzle_mysql\drizzle.config.js'
Warning Drizzle Studio is currently in Beta. If you find anything that is not working as expected or should be improved, feel free to create an issue on GitHub: https://github.com/drizzle-team/drizzle-kit-mirror/issues/new or write to us on Discord: https://discord.gg/WcRKz2FFxN
Drizzle Studio is up and running on https://local.drizzle.studio
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
meta
0000_messy_odin.sql
schema.js
node_modules
.env
drizzle.config.js
package-lock.json
package.json

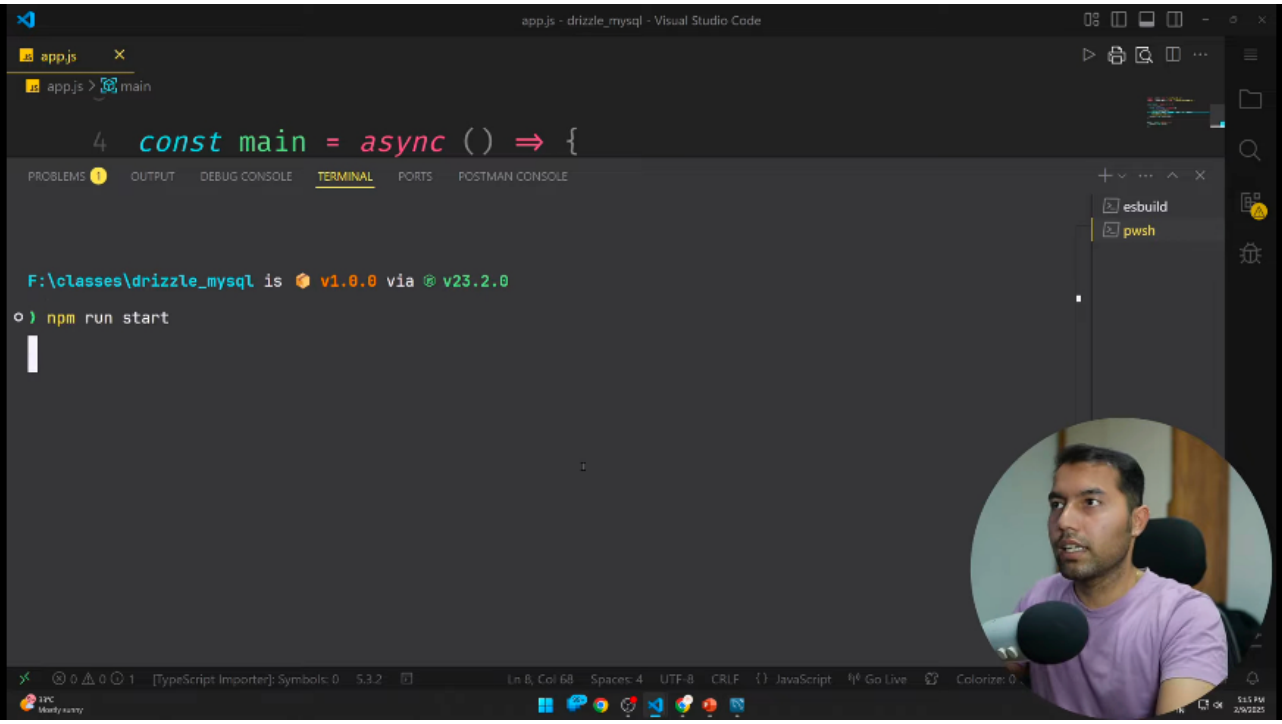

CRUD OPER.
"version": "1.0.0",
"main": "index.js",
"db:studio": "drizzle-kit studio"
"keywords": [],
"author": "",
"license": "ISC",
"description": "",
"dependencies": {

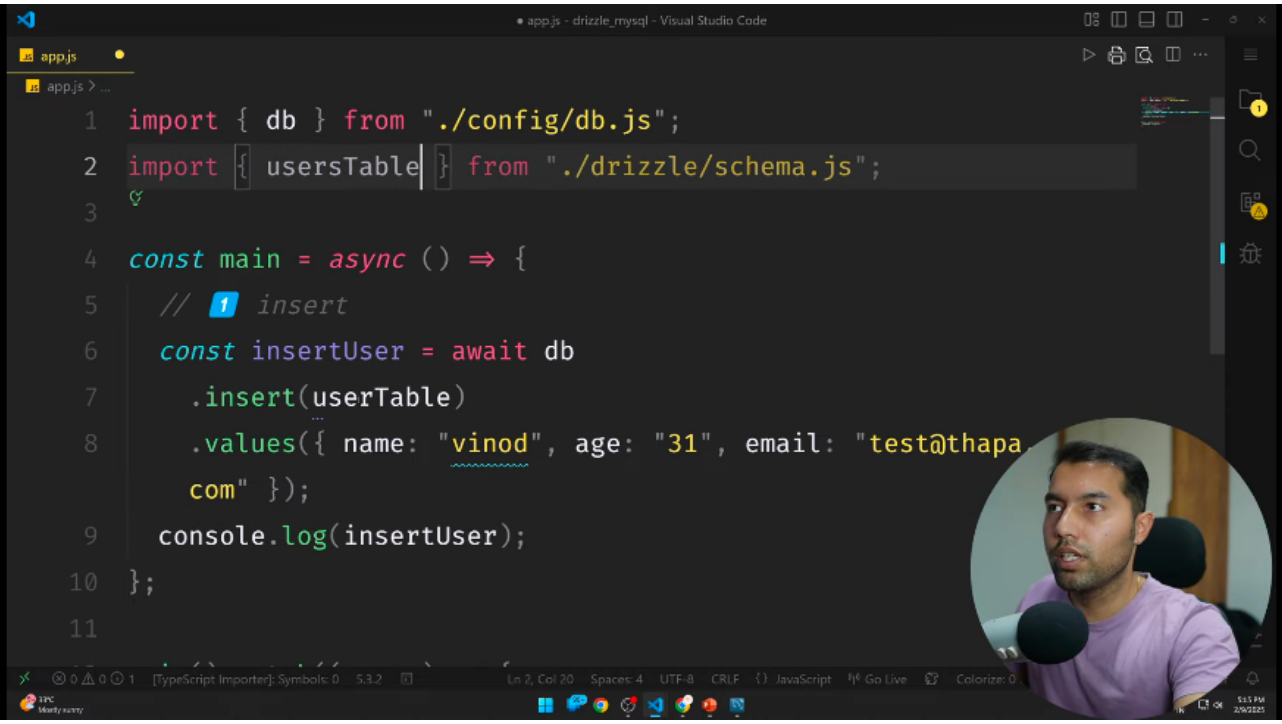

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main
import { db } from "./config/db.js";
import { userTable } from "./drizzle/schema.js";
const main = async () => {
// 1 insert
}
main().catch((error) => {
console.log(error);
});

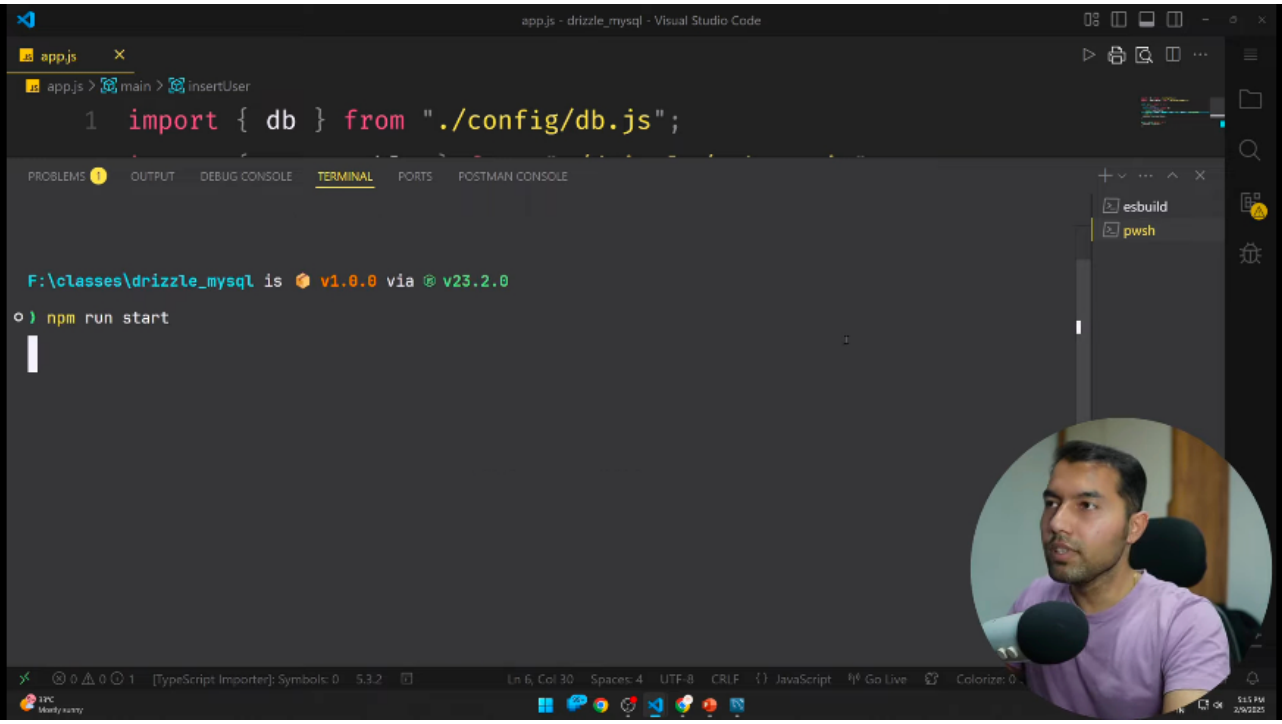

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main > insertUser
import { userTable } from "./drizzle/schema.js";
const main = async () => {
// 1 insert
const insertUser = await db.insert().
}
INSERT INTO
table_name (column1, column2, ...)
VALUES (value1, value2, value3, ...);

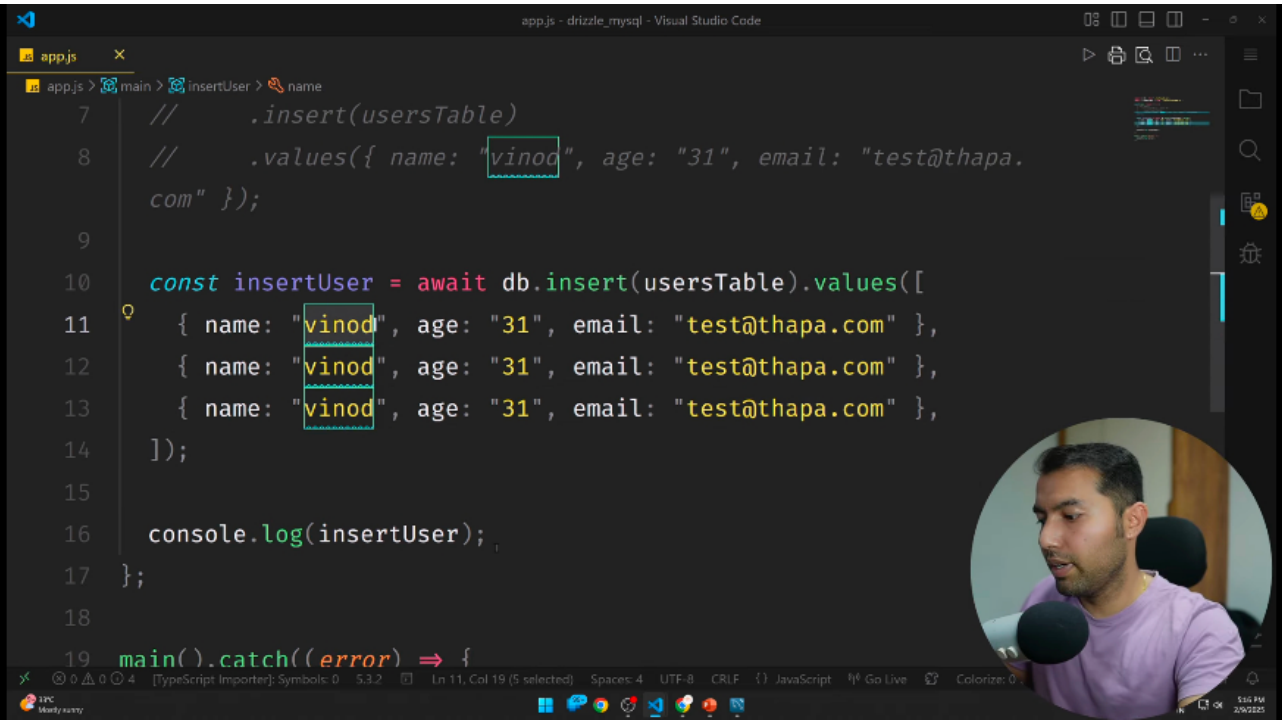

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main
const main = async () => {
// 1 insert
const insertUser = await db
.insert(userTable)
.values({ name: "vinod", age: "31", email:
"test@thapa.com" });
};
main().catch((error) => {
console.log(error);

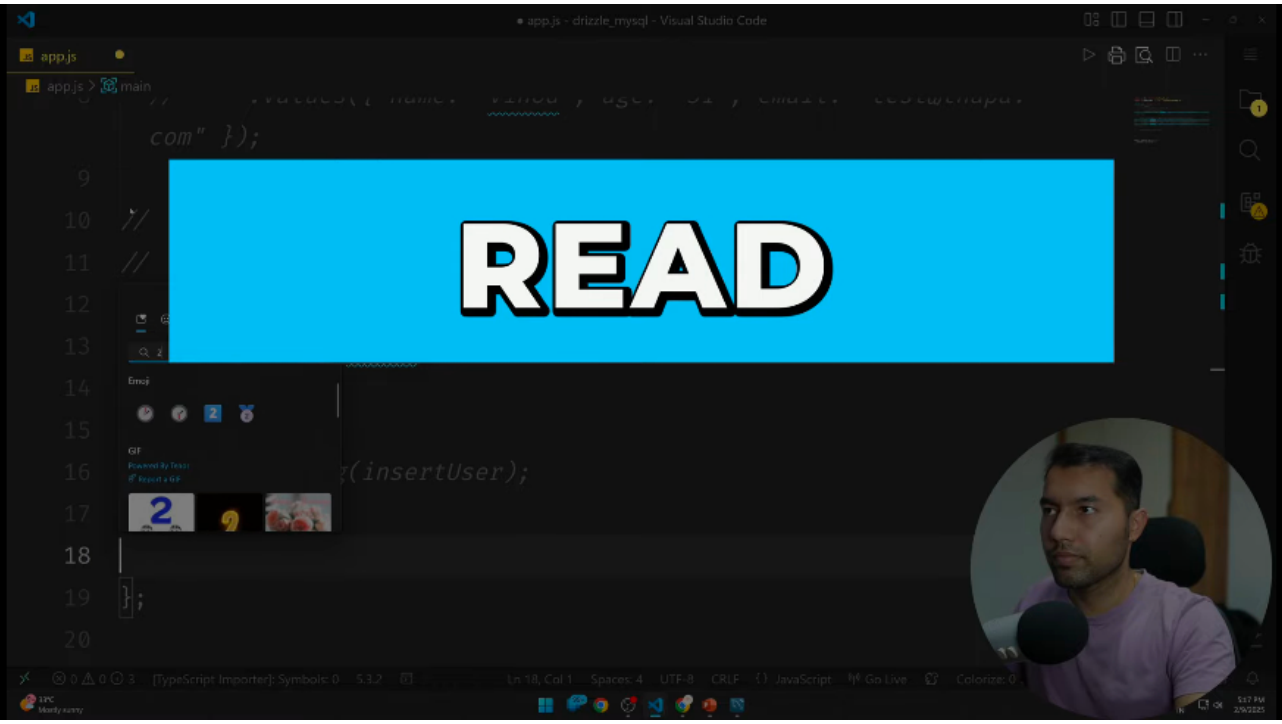

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main
const main = async () => {
// 1 insert
const insertUser = await db
.insert(userTable)
.values({ name: "vinod", age: "31", email:
"test@thapa.com" });
console.log(insertUser);
};
main().catch((error) => {
console.log(error);

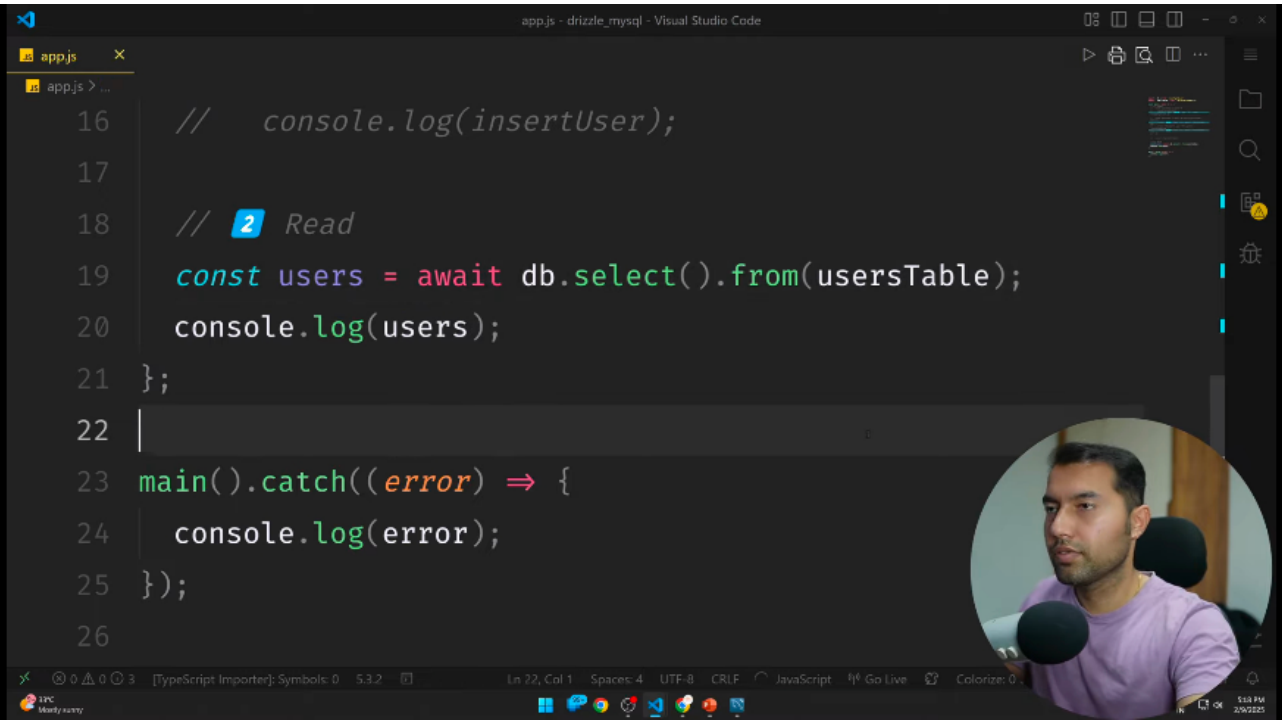

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main
4 const main = async () => {
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
esbuild
pwsh
F:\classes\drizzle_mysql is v1.0.0 via v23.2.0
❯ npm run start

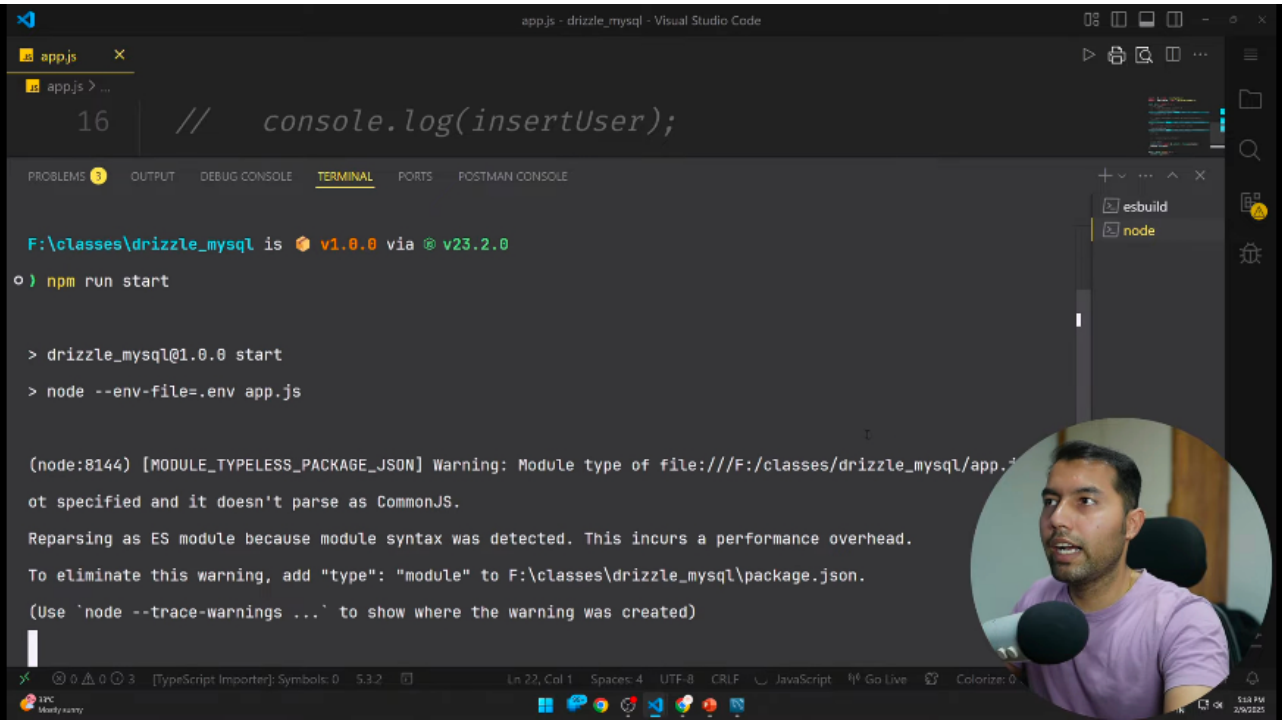

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > ...
1 import { db } from "./config/db.js";
2 import { usersTable } from "./drizzle/schema.js";
3
4 const main = async () => {
5 // insert
6 const insertUser = await db
7 .insert(userTable)
8 .values({ name: "vinod", age: "31", email: "test@thapa.com" });
9 console.log(insertUser);
10 };
11


app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main > insertUser
1 import { db } from "./config/db.js";
PROBLEMS 1
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
esbuild
pwsh
F:\classes\drizzle_mysql is v1.0.0 via v23.2.0
❯ npm run start
Ln 6, Col 30
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live
Colorize: 0

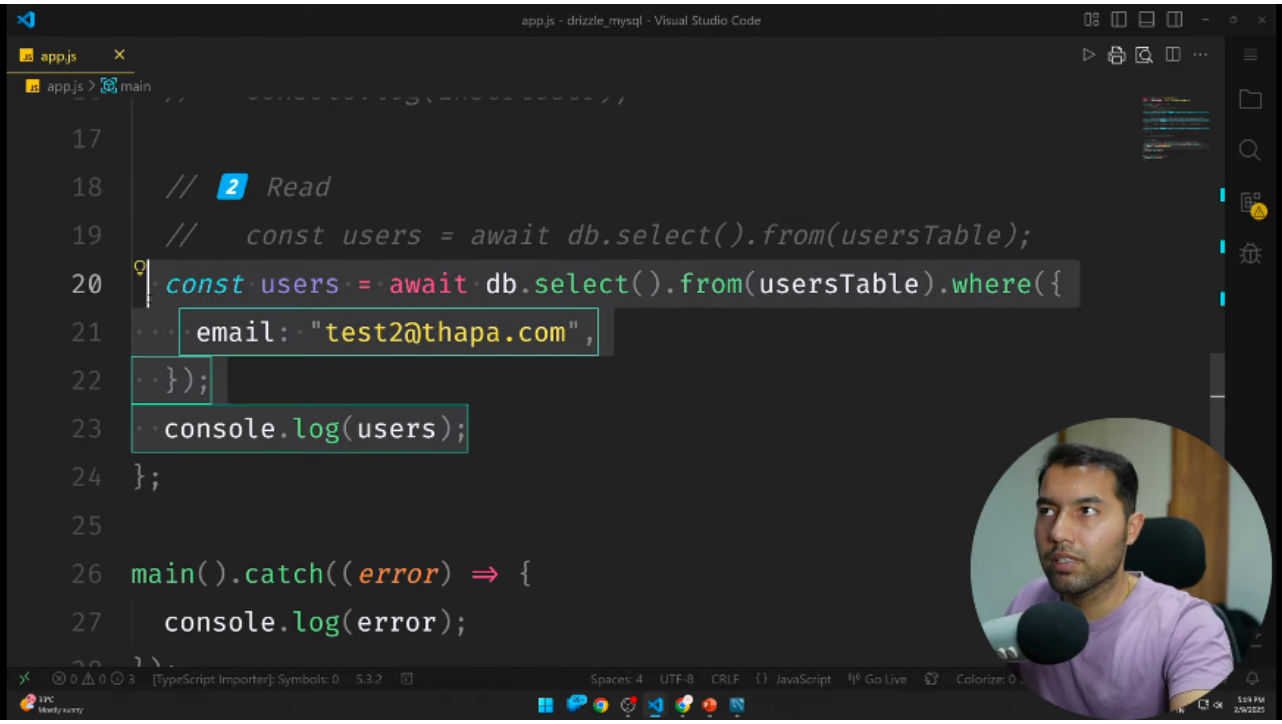

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main > insertUser > name
// .insert(usersTable)
// .values({ name: "vinod", age: "31", email: "test@thapa.
com" });
const insertUser = await db.insert(usersTable).values([
{ name: "vinod", age: "31", email: "test@thapa.com" },
{ name: "vinod", age: "31", email: "test@thapa.com" },
{ name: "vinod", age: "31", email: "test@thapa.com" },
]);
console.log(insertUser);
};
main().catch((error) => {


READ

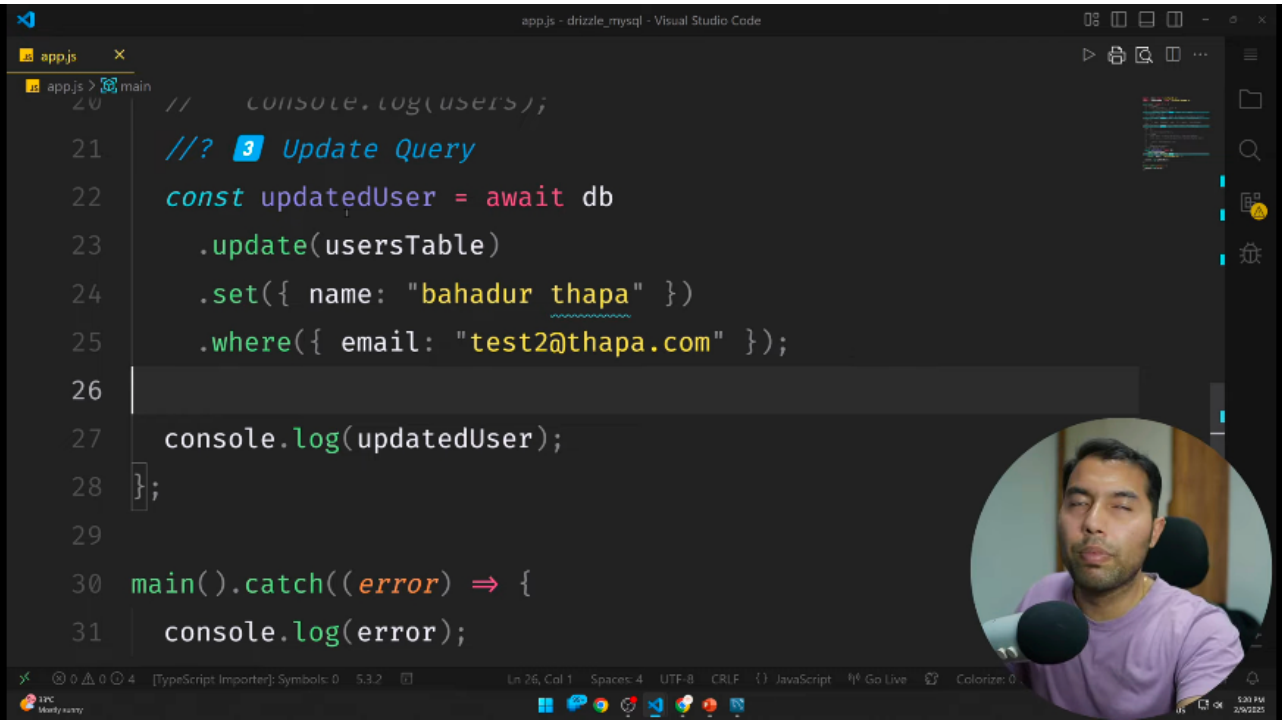

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > ...
16 // console.log(insertUser);
17
18 // 2 Read
19 const users = await db.select().from(usersTable);
20 console.log(users);
21 };
22
23 main().catch((error) => {
24 console.log(error);
25 });
26

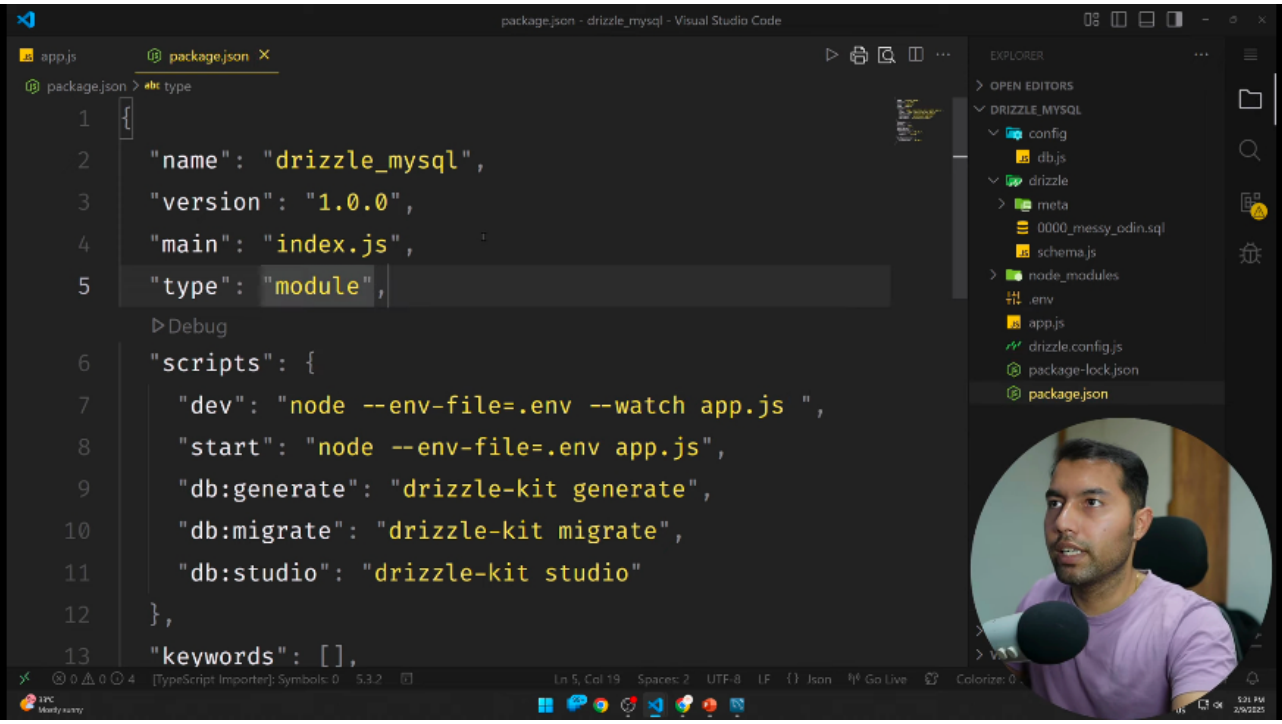

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > ...
16 // console.log(insertUser);
PROBLEMS 3
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
POSTMAN CONSOLE
esbuild
node
F:\classes\drizzle_mysql is v1.0.0 via v23.2.0
❯ npm run start
> drizzle_mysql@1.0.0 start
> node --env-file=.env app.js
(node:8144) [MODULE_TYPELESS_PACKAGE_JSON] Warning: Module type of file:///F:/classes/drizzle_mysql/app.
ot specified and it doesn't parse as CommonJS.
Reparsing as ES module because module syntax was detected. This incurs a performance overhead.
To eliminate this warning, add "type": "module" to F:\classes\drizzle_mysql\package.json.
(Use `node --trace-warnings ...` to show where the warning was created)
Ln 22, Col 1
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live

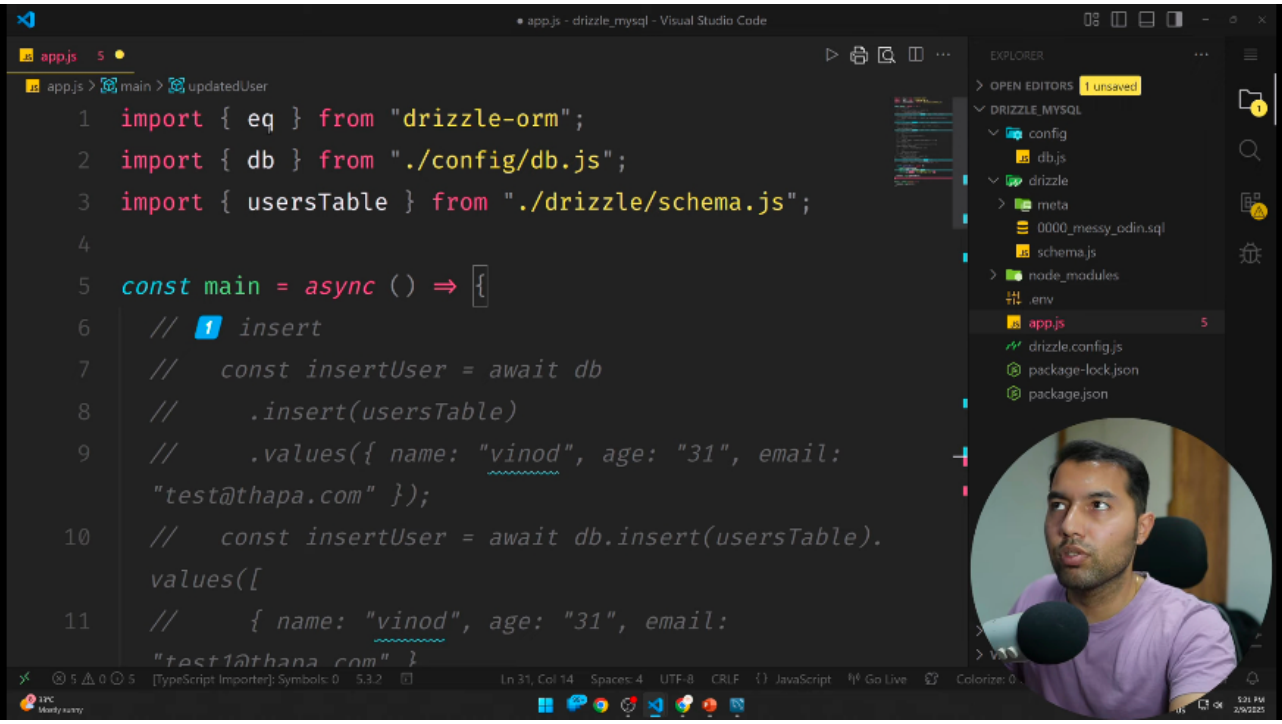
```
(Use `node --trace-warnings ...` to show where the warning was cr
[
  { id: 1, name: 'vinod', age: 31, email: 'test@thapa.com' },
  { id: 2, name: 'vinod', age: 31, email: 'test1@thapa.com' },
  { id: 3, name: 'bahadur', age: 31, email: 'test2@thapa.com' },
  { id: 4, name: 'thapa', age: 31, email: 'test3@thap
]
```

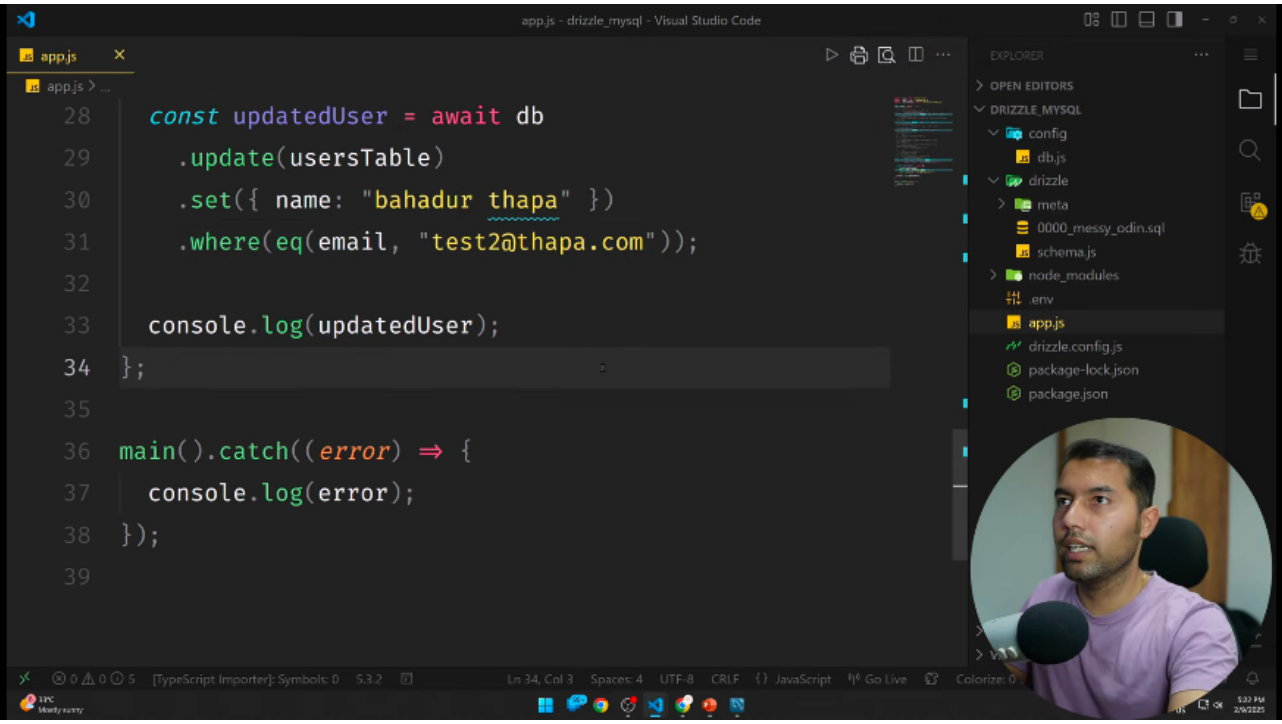

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main
// 2 Read
// const users = await db.select().from(usersTable);
const users = await db.select().from(usersTable).where({
email: "test2@thapa.com",
});
console.log(users);
};
main().catch((error) => {
console.log(error);

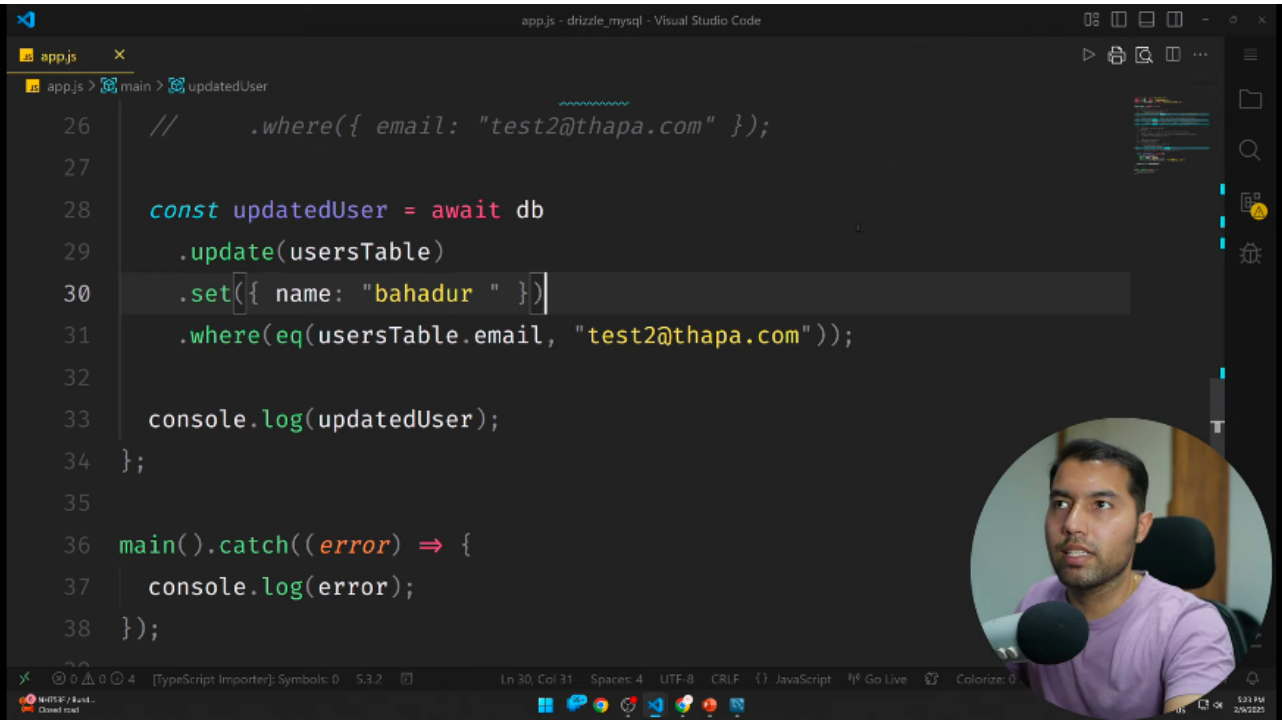

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main
// 2 Read
// console.log(users);
};
main(
UPDATE

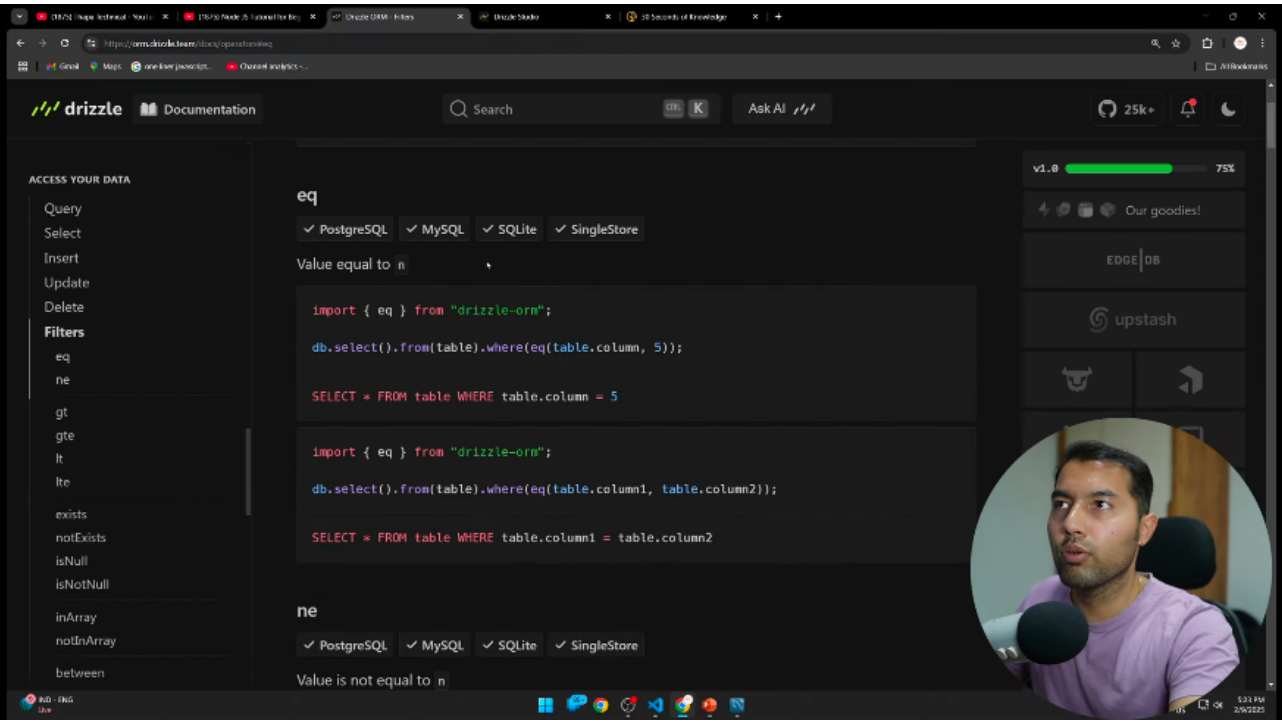

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main
// console.log(users);
//? 3 Update Query
const updatedUser = await db
.update(usersTable)
.set({ name: "bahadur thapa" })
.where({ email: "test2@thapa.com" });
console.log(updatedUser);
};
main().catch((error) => {
console.log(error);

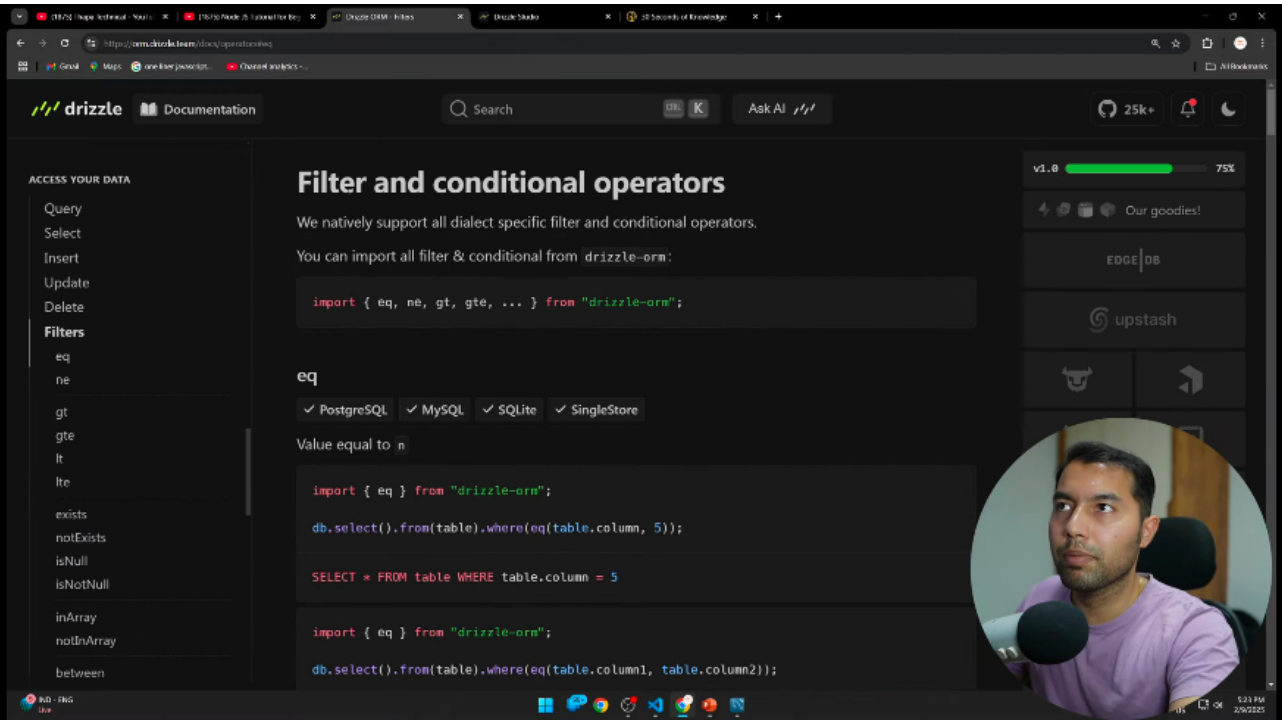

package.json - drizzle_mysql - Visual Studio Code
app.js
package.json
package.json > type
{
"name": "drizzle_mysql",
"version": "1.0.0",
"main": "index.js",
"type": "module",
Debug
"scripts": {
"dev": "node --env-file=.env --watch app.js ",
"start": "node --env-file=.env app.js",
"db:generate": "drizzle-kit generate",
"db:migrate": "drizzle-kit migrate",
"db:studio": "drizzle-kit studio"
},
"keywords": [],
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
meta
0000_messy_odin.sql
schema.js
node_modules
.env
app.js
drizzle.config.js
package-lock.json
package.json
Ln 5, Col 19
Spaces: 2
UTF-8
LF
Json
Go Live

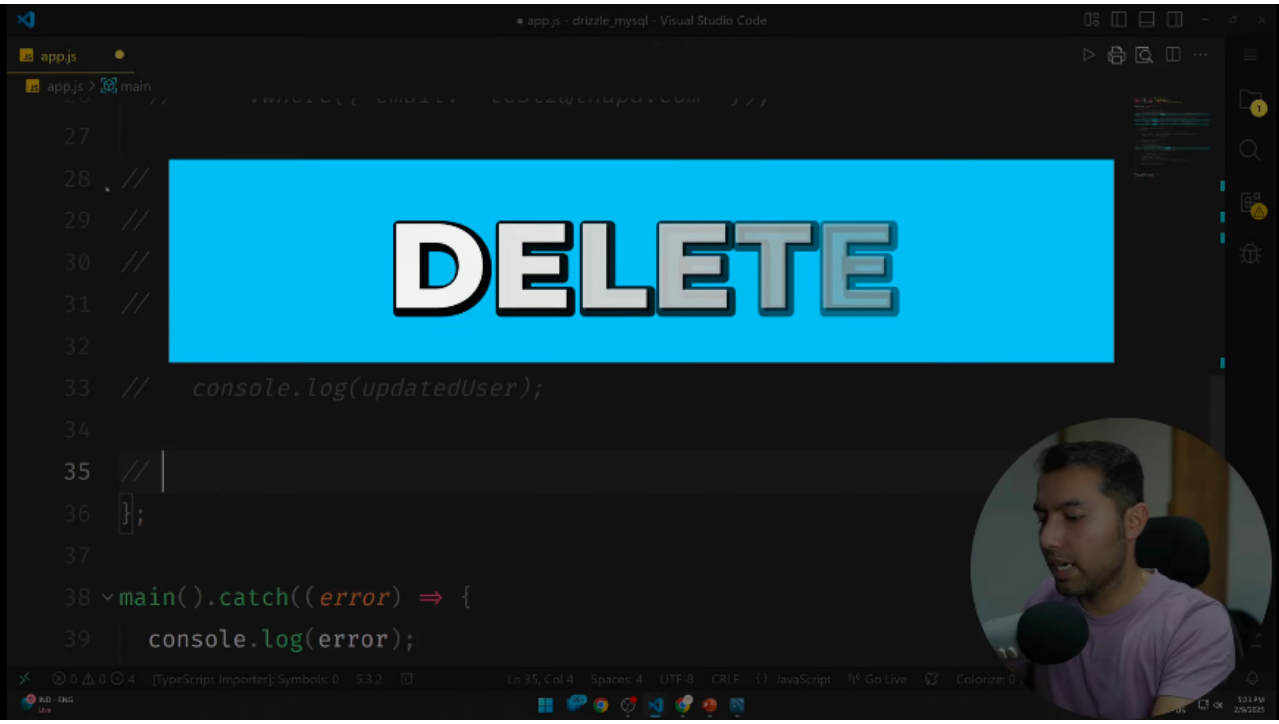

app.js - drizzle_mysql - Visual Studio Code
app.js 5
app.js > main > updatedUser
1 import { eq } from "drizzle-orm";
2 import { db } from "./config/db.js";
3 import { usersTable } from "./drizzle/schema.js";
4
5 const main = async () => {
6 // 1 insert
7 // const insertUser = await db
8 // .insert(usersTable)
9 // .values({ name: "vinod", age: "31", email:
"test@thapa.com" });
10 // const insertUser = await db.insert(usersTable).
values([
11 // { name: "vinod", age: "31", email:
EXPLORER
OPEN EDITORS 1 unsaved
DRIZZLE_MYSQL
config
db.js
drizzle
meta
0000_messy_odin.sql
schema.js
node_modules
.env
app.js 5
drizzle.config.js
package-lock.json
package.json
Ln 31, Col 14 Spaces: 4 UTF-8 CRLF JavaScript Go Live

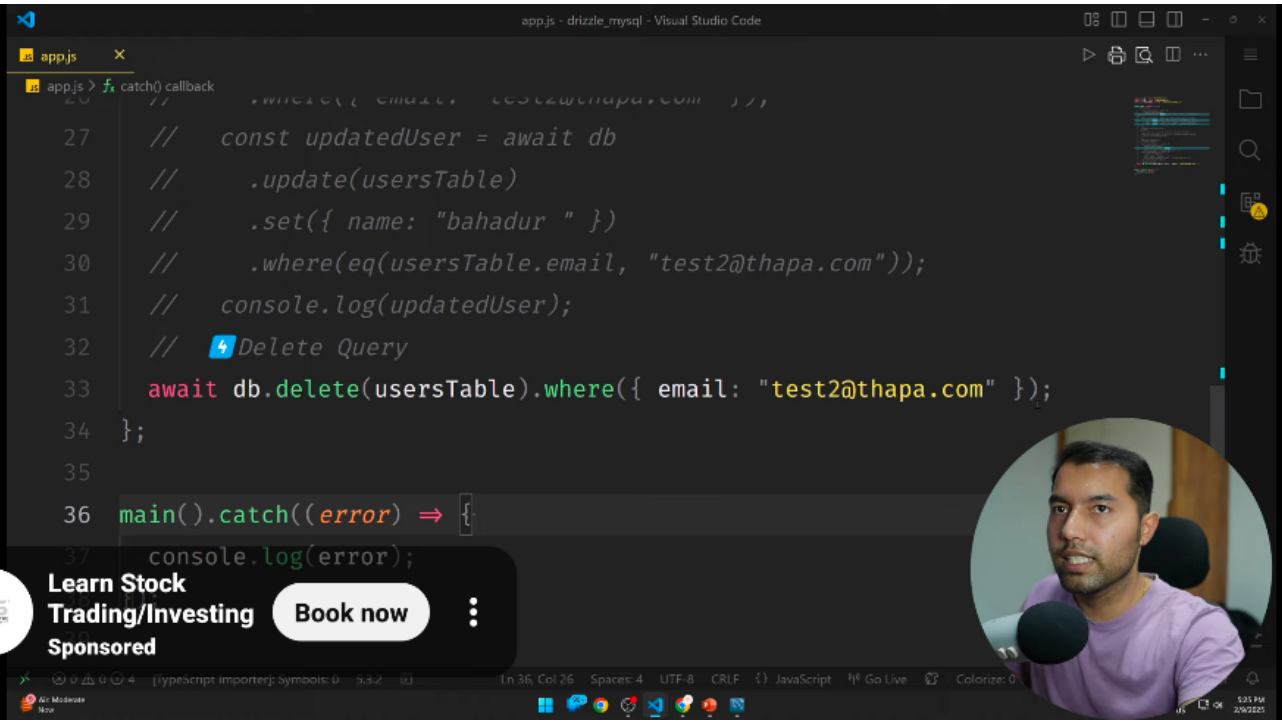

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > ...
28 const updatedUser = await db
29 .update(usersTable)
30 .set({ name: "bahadur thapa" })
31 .where(eq(email, "test2@thapa.com"));
32
33 console.log(updatedUser);
34 };
35
36 main().catch((error) => {
37 console.log(error);
38 });
39
EXPLORER
OPEN EDITORS
DRIZZLE_MYSQL
config
db.js
drizzle
meta
0000_messy_odin.sql
schema.js
node_modules
.env
app.js
drizzle.config.js
package-lock.json
package.json
Ln 34, Col 3
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live

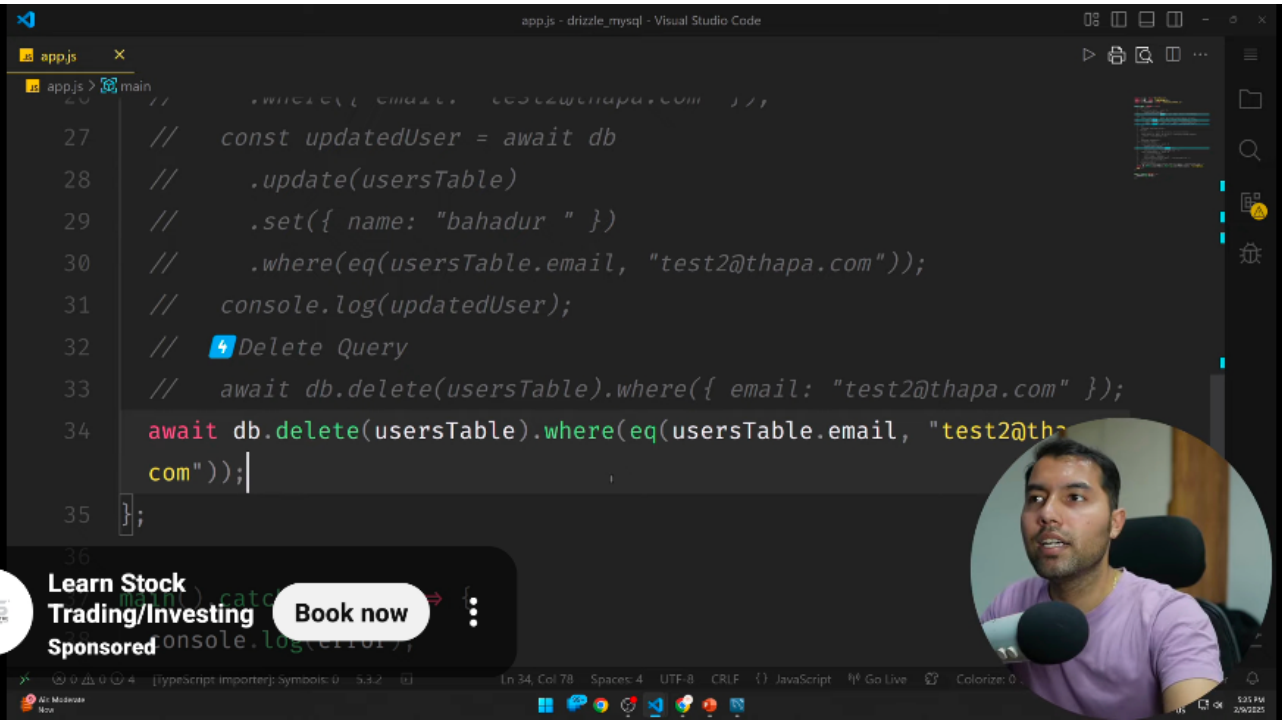

app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main > updatedUser
26 // .where({ email: "test2@thapa.com" });
27
28 const updatedUser = await db
29 .update(usersTable)
30 .set({ name: "bahadur " })
31 .where(eq(usersTable.email, "test2@thapa.com"));
32
33 console.log(updatedUser);
34 };
35
36 main().catch((error) => {
37 console.log(error);
38 });


drizzle
Documentation
Search
Ask AI
25k+
ACCESS YOUR DATA
Query
Select
Insert
Update
Delete
Filters
eq
ne
gt
gte
lt
lte
exists
notExists
isNull
isNotNull
inArray
notInArray
between
eq
PostgreSQL
MySQL
SQLite
SingleStore
Value equal to n
import { eq } from "drizzle-orm";
db.select().from(table).where(eq(table.column, 5));
SELECT * FROM table WHERE table.column = 5
import { eq } from "drizzle-orm";
db.select().from(table).where(eq(table.column1, table.column2));
SELECT * FROM table WHERE table.column1 = table.column2
ne
PostgreSQL
MySQL
SQLite
SingleStore
Value is not equal to n
v1.0
75%
Our goodies!
EDGE DB
upstash


drizzle
Documentation
Search
Ask AI
25k+
ACCESS YOUR DATA
Query
Select
Insert
Update
Delete
Filters
eq
ne
gt
gte
lt
lte
exists
notExists
isNull
isNotNull
inArray
notInArray
between
Filter and conditional operators
We natively support all dialect specific filter and conditional operators.
You can import all filter & conditional from drizzle-orm:
import { eq, ne, gt, gte, ... } from "drizzle-orm";
eq
✓ PostgreSQL ✓ MySQL ✓ SQLite ✓ SingleStore
Value equal to n
import { eq } from "drizzle-orm";
db.select().from(table).where(eq(table.column, 5));
SELECT * FROM table WHERE table.column = 5
import { eq } from "drizzle-orm";
db.select().from(table).where(eq(table.column1, table.column2));
v1.0
75%
Our goodies!
EDGE DB
upstash


app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main
DELETE
// console.log(updatedUser);
main().catch((error) => {
console.log(error);


app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > catch() callback
// .where({ email: "test2@thapa.com" });
// const updatedUser = await db
// .update(usersTable)
// .set({ name: "bahadur " })
// .where(eq(usersTable.email, "test2@thapa.com"));
// console.log(updatedUser);
// Delete Query
await db.delete(usersTable).where({ email: "test2@thapa.com" });
};
main().catch((error) => {
console.log(error);
Learn Stock Trading/Investing
Book now
Sponsored


app.js - drizzle_mysql - Visual Studio Code
app.js
app.js > main
// .where({ email: "test2@thapa.com" });
// const updatedUser = await db
// .update(usersTable)
// .set({ name: "bahadur " })
// .where(eq(usersTable.email, "test2@thapa.com"));
// console.log(updatedUser);
// Delete Query
// await db.delete(usersTable).where({ email: "test2@thapa.com" });
await db.delete(usersTable).where(eq(usersTable.email, "test2@th
com"));
};
Learn Stock Trading/Investing
Sponsored
Book now
Ln 34, Col 78
Spaces: 4
UTF-8
CRLF
JavaScript
Go Live

Top 10 ORM Interview Questions and Answers

Updated 10 Oct 2025

Asked in **Fynd**
6d ago

Q. Why ORMs are used? Write SQL query for two-three questions!

Ans. ORMs are used to map object-oriented code to relational databases.

- ORMs simplify database interactions by abstracting away SQL syntax.
- ORMs allow developers to work with objects instead of raw SQL queries.
- ORMs can handle complex relationships between database tables.
- ORMs can improve code readability and maintainability.
- Example ORMs include Hibernate for Java and Entity Framework for .NET.

Answered by AI

View all 2 answers Share

Asked in **Vista Foundation**
4d ago

Q. What are the benefits of using an ORM tool?

Ans. ORM tools provide abstraction, simplify database operations, improve code maintainability, and enhance productivity.

- ORM tools abstract the database layer, allowing developers to work with objects instead of writing complex SQL queries.
- They simplify database operations by automatically generating SQL queries based on object-oriented code.
- ORM tools handle database connections, transactions, and caching, reducing the amount of boilerplate code.
- They improve code maintainability by providing a clear separation between business logic and data access code.
- ORM tools enhance productivity by providing built-in features like query optimization, schema migration, and object-relational mapping.
- Examples of popular ORM tools include Hibernate for Java, Entity Framework for .NET, and Sequelize for Node.js.

Answered by AI

View all 3 answers Share

Asked in **Marlabs**
2d ago

Q. What is the difference between the code-first and database-first approaches?

Ans. Code first approach involves creating the database schema based on the code, while database first approach involves creating the code based on an existing database schema.

- Code first approach focuses on defining the entities and their relationships in code, and then generating the database schema from the code.
- Database first approach involves designing the database schema first, and then generating the code based on the schema.
- Code first approach provides more control over the database schema and allows for easier versioning and migration.
- Database first approach is useful when working with an existing database or when the database schema is already defined.
- Code first approach is commonly used in Object-Relational Mapping (ORM) frameworks like Entity Framework.
- Database first approach is commonly used when working with legacy systems or when the database design is critical.

Answered by AI

View all 2 answers Share

Asked in **Markytics**
5d ago

Q. What is Orm, and what fundamentals of orm

Ans. ORM stands for Object-Relational Mapping. It is a programming technique for converting data between incompatible type systems in object-oriented programming languages.

- ORM helps developers work with databases using object...read more

Add your answer Share

Top Interview Questions for Related Skills

SQL Interview Questions and Answers
250 Questions

DBMS Interview Questions and Answers
100 Questions

MySQL Interview Questions and Answers
50 Questions

Oracle Interview Questions and Answers
50 Questions

Triggers Interview Questions and Answers
50 Questions

Interview Questions of ORM Related Designations

Software Engineer Interview Questions and Answers
9.5k Questions

Asked in **Markytics**
5d ago

Q. What is Orm, and what fundamentals of orm

Ans. ORM stands for Object-Relational Mapping. It is a programming technique for converting data between incompatible type systems in object-oriented programming languages.

- ORM helps developers work with databases using objects instead of SQL queries
- It simplifies data manipulation and reduces the amount of code needed to interact with a database
- ORM frameworks like SQLAlchemy in Python provide tools for mapping objects to database tables

Answered by AI

Add your answer Share

Are these interview questions helpful? Yes No

Asked in **Netscribes**
4d ago

Q. What is ORM, and what are some best practices?

Ans. ORM stands for Object-Relational Mapping, a programming technique for converting data between incompatible type systems in object-oriented programming languages.

- ORM helps in mapping objects to database tables and vice versa
- Avoid using ORM for complex queries as it can lead to performance issues
- Use lazy loading to optimize performance by loading data only when needed

Answered by AI

Add your answer Share

Asked in **Deloitte**
1d ago

Q. What is your understanding of ERM and ORM?

Ans. ERM stands for Enterprise Risk Management, which involves identifying, assessing, and managing risks across an organization. ORM stands for Operational Risk Management, which focuses on managing risks related to day-to-day operations.

- ERM involves a holistic approach to managing risks at an organizational level.
- ORM focuses on identifying and mitigating risks that can impact daily operations.
- ERM helps in aligning risk management strategies with overall business objectives.
- ORM includes activities such as risk assessment, risk monitoring, and risk reporting.
- Examples of ERM tools include risk registers, risk heat maps, and risk appetite frameworks.
- Examples of ORM tools include incident reporting systems, key risk indicators, and control self-assessments.

Answered by AI

Add your answer Share

Share interview questions and help millions of jobseekers 🌟

Share interview questions

Asked in **Bristlefront Software**
4d ago

Q. What is the difference between ORM and JPA?

Ans. JPA is a specification for ORM in Java, while ORM is a technique for mapping objects to relational databases.

- JPA is a Java specification for ORM, while ORM is a technique for mapping objects to relational databases
- ORM is...read more

Add your answer Share

Asked in **KnowCross Solutions**
2d ago

Top Interview Questions for Related Skills

SQL Interview Questions and Answers
250 Questions

DBMS Interview Questions and Answers
100 Questions

MySQL Interview Questions and Answers
50 Questions

Oracle Interview Questions and Answers
50 Questions

Triggers Interview Questions and Answers
50 Questions

Interview Questions of ORM Related Designations

Software Engineer Interview Questions and Answers
9.5k Questions

Interview Tips & Stories

Ace your next interview with expert advice and inspiring stories

Explore community

Add Questions

Asked in **Bristlefront Software**
4d ago

Q. What is the difference between ORM and JPA?

Ans. JPA is a specification for ORM in Java, while ORM is a technique for mapping objects to relational databases.

- JPA is a Java specification for ORM, while ORM is a technique for mapping objects to relational databases
- ORM is a general concept, while JPA is a specific implementation of ORM for Java
- JPA provides a set of interfaces and annotations for mapping Java objects to relational databases
- ORM frameworks like Hibernate, TopLink, and iBatis can be used with JPA

Answered by AI

Add your answer Share

Asked in **KnowCross Solutions**
2d ago

Q. What is orm. What orm models have you worked on

Ans. ORM stands for Object-Relational Mapping. It is a technique to map database tables to classes in object-oriented programming.

- ORM allows developers to work with databases using object-oriented programming concepts.
- ORM models I have worked on include Hibernate, Entity Framework, and Sequelize.
- ORM helps to reduce the amount of boilerplate code required to interact with databases.
- ORM provides a layer of abstraction between the application and the database, making it easier to switch databases or make changes to the database schema.
- ORM can also help to improve performance by reducing the number of database queries required.

Answered by AI

Add your answer Share

Asked in **OpenDoors Business Solutions**
6d ago

Q. What are the relationships in Eloquent ORM?

Ans. Relationships in Eloquent ORM define how different database tables are related to each other.

- Eloquent ORM supports relationships like hasOne, hasMany, belongsTo, belongsToMany, morphOne, morphMany, morphTo, and morphToMany.
- Example: A User model may have a hasMany relationship with a Post model, where each user can have multiple posts.
- Example: A Comment model may belongTo a Post model, where each comment belongs to a single post.

Answered by AI

Add your answer Share

Asked in **LinkSoft Technologies**
3d ago

Q. Rails features realted to ORM

Ans. Rails features related to ORM include ActiveRecord, associations, validations, and callbacks.

- ActiveRecord provides ORM functionality in Rails
- Associations allow defining relationships between models
- Validations ensure data integrity and consistency
- Callbacks enable executing code at specific points in the object lifecycle

Answered by AI

Add your answer Share

🧠 Basic Drizzle ORM Interview Questions and Answers

1. What is Drizzle ORM?

Answer:
Drizzle ORM is a TypeScript-first, lightweight, and SQL-focused ORM for modern JavaScript frameworks like Next.js, Node.js, Bun, and Deno.
It provides type safety, schema generation, and query building directly in TypeScript — making database interactions safe and developer-friendly.

---

2. Why use Drizzle ORM over Prisma or Sequelize?

Answer:
Drizzle ORM is preferred because:

- ⚡ Lightweight and faster build times.
- 💬 SQL-like syntax (closer to actual SQL).
- 🧩 Type-safe migrations and queries.
- 🧠 No runtime overhead — queries are generated at compile time.
- 🛠️ Works perfectly with Edge environments (e.g., Vercel, Cloudflare).

---

3. What databases are supported by Drizzle ORM?

Answer:
Drizzle supports:

- PostgreSQL
- MySQL
- SQLite
- Neon, Turso, PlanetScale (serverless DBs)
- LibSQL

---

4. How do you define a table in Drizzle ORM?

Answer:
In Drizzle, you define tables using TypeScript schemas.

Example:

typescript Copy code

```typescript
import { pgTable, serial, varchar, integer } from "drizzle-orm/pg-core";

export const users = pgTable("users", {
  id: serial("id").primaryKey(),
  name: varchar("name", { length: 100 }),
  age: integer("age"),
});
```

---

5. How to insert data using Drizzle ORM?

5. How to insert data using Drizzle ORM?

Answer:

typescript Copy code

```typescript
await db.insert(users).values({
  name: "Ankit Pawar",
  age: 22,
});
```

6. How to fetch all records using Drizzle ORM?

Answer:

typescript Copy code

```typescript
const allUsers = await db.select().from(users);
```

7. How to use `where` conditions in Drizzle ORM?

Answer:

typescript Copy code

```typescript
import { eq } from "drizzle-orm";

const result = await db.select().from(users).where(eq(users.name, "Ankit P
```

8. What is `eq` in Drizzle ORM?

Answer:

`eq` is a comparison operator used in queries (similar to SQL's `=`).

Drizzle provides several such helpers:

- `eq()` → equals
- `gt()` → greater than
- `lt()` → less than
- `and()`, `or()` → logical operators

9. How do you update data using Drizzle ORM?

Answer:

typescript Copy code

```typescript
await db.update(users)
  .set({ age: 23 })
  .where(eq(users.name, "Ankit Pawar"));
```

10. How to delete a record in Drizzle ORM?

Answer:

```typescript
await db.delete(users).where(eq(users.id, 1));
```

⚙️ Intermediate Drizzle ORM Questions

11. What are Migrations in Drizzle ORM?

Answer:
Migrations are used to track and apply schema changes (like creating tables or altering columns).
Drizzle provides a built-in migration system using:

```bash
drizzle-kit generate
drizzle-kit push
```

12. How do you generate migrations in Drizzle?

Answer:
Run this command in your terminal:

```bash
npx drizzle-kit generate
```

It automatically creates migration files based on your schema definitions.

13. What is the `drizzle.config.ts` file?

Answer:
This file defines your Drizzle ORM setup:

```typescript
export default {
  schema: "./src/db/schema.ts",
  out: "./drizzle",
  driver: "pg",
  dbCredentials: {
    connectionString: process.env.DATABASE_URL!,
  },
};
```

It's used by Drizzle Kit to connect to your database and manage migrations.

14. How to connect Drizzle ORM with Neon Database (Postgres)?

Answer:

typescript Copy code

```typescript
import { drizzle } from "drizzle-orm/neon-http";
import { neon } from "@neondatabase/serverless";

const sql = neon(process.env.DATABASE_URL!);
export const db = drizzle(sql);
```

15. How to perform JOIN operations in Drizzle ORM?

Answer:

typescript Copy code

```typescript
import { eq } from "drizzle-orm";
import { posts, users } from "./schema";

const result = await db
  .select()
  .from(posts)
  .leftJoin(users, eq(posts.userId, users.id));
```

16. What are some advantages of using Drizzle ORM in Next.js projects?

Answer:

- Works perfectly in Edge runtimes (like Vercel Edge).
- Tree-shakable and lightweight.
- Type-safe migrations and queries.
- Can run directly in API routes or server actions.

17. How does Drizzle ensure type safety?

Answer:

Drizzle generates TypeScript types directly from schema definitions, ensuring:

- No mismatched column names or data types.
- Autocompletion in VS Code.
- Compile-time error checking.

18. Can you write raw SQL in Drizzle ORM?

Answer:

Yes, Drizzle allows raw SQL queries when needed:

typescript Copy code

```typescript
import { sql } from "drizzle-orm";
```

18. Can you write raw SQL in Drizzle ORM?

Answer:
Yes, Drizzle allows raw SQL queries when needed:

typescript Copy code

```typescript
import { sql } from "drizzle-orm";

await db.execute(sql`SELECT * FROM users WHERE age > 20`);
```

---

19. How to count records in Drizzle ORM?

Answer:

typescript Copy code

```typescript
import { count } from "drizzle-orm";

const result = await db.select({ total: count() }).from(users);
```

---

20. What is the purpose of `drizzle-kit`?

Answer:
`drizzle-kit` is a CLI tool used to:

- Generate and run migrations
- Sync schemas between code and database
- Visualize schemas

💡 Bonus: HR/Project-based Questions

21. Where did you use Drizzle ORM in your project?

Answer (Example for You):
I used Drizzle ORM in my AI-assisted waste verification project with Next.js and Neon database.
It helped manage database schema easily, run migrations, and perform type-safe queries for storing waste verification records and user data.

---

22. What challenges did you face while using Drizzle ORM?

Answer:

- Setting up Neon database connection for Edge functions.
- Understanding migration versioning.
- Handling relations (one-to-many) in early versions.

But once configured, Drizzle ORM was fast, stable, and easier to maintain.

---

23. How does Drizzle ORM differ from Prisma?

💡 Bonus: HR/Project-based Questions

21. Where did you use Drizzle ORM in your project?

Answer (Example for You):
I used Drizzle ORM in my AI-assisted waste verification project with Next.js and Neon database.
It helped manage database schema easily, run migrations, and perform type-safe queries for storing waste verification records and user data.

---

22. What challenges did you face while using Drizzle ORM?

Answer:

- Setting up Neon database connection for Edge functions.
- Understanding migration versioning.
- Handling relations (one-to-many) in early versions.

But once configured, Drizzle ORM was fast, stable, and easier to maintain.

---

23. How does Drizzle ORM differ from Prisma?

Feature	Drizzle ORM	Prisma
Type Safety	✅ Excellent	✅ Excellent
Migrations	✅ Built-in (Type-based)	✅ Schema-based
Performance	⚡ Fast (compile-time)	🐢 Slightly slower
Syntax	SQL-like	Model-based
File Size	Smaller	Larger

---

24. What is a relation in Drizzle ORM?

Answer:
Relations define links between tables (like foreign keys).
Example:

typescript Copy code

```typescript
import { relations } from "drizzle-orm";

export const userRelations = relations(users, ({ many }) => ({
  posts: many(posts),
}));
```

---

25. How can you debug Drizzle ORM queries?

Answer:
You can log generated SQL queries by using:

23. How does Drizzle ORM differ from Prisma?

Feature	Drizzle ORM	Prisma
Type Safety	✅ Excellent	✅ Excellent
Migrations	✅ Built-in (Type-based)	✅ Schema-based
Performance	⚡ Fast (compile-time)	🐢 Slightly slower
Syntax	SQL-like	Model-based
File Size	Smaller	Larger

---

24. What is a relation in Drizzle ORM?

Answer:
Relations define links between tables (like foreign keys).
Example:

typescript Copy code

```typescript
import { relations } from "drizzle-orm";

export const userRelations = relations(users, ({ many }) => ({
  posts: many(posts),
}));
```

---

25. How can you debug Drizzle ORM queries?

Answer:
You can log generated SQL queries by using:

typescript Copy code

```typescript
console.log(db.$query);
```

Or use Drizzle Studio (coming soon) for visual schema inspection.

Would you like me to prepare a short revision PDF (Drizzle ORM Interview Cheatsheet for Freshers) — summarizing these Q&As with examples and commands (perfect for last-minute prep)?